
Modelos generativos

PID_00290597

Jordi de la Torre Gallart

Tiempo mínimo de dedicación recomendado: 6 horas



Jordi de la Torre Gallart

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

Cómo citar este recurso de aprendizaje con el estilo Harvard:

De la Torre Gallart, J. (2024). *Modelos generativos*. [Recurso de aprendizaje textual]. 1.ª ed. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024
© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)
Av. Tibidabo, 39-43, 08035 Barcelona
Autoría: Jordi de la Torre Gallart
Producción: FUOC
Todos los derechos reservados

Ninguna parte de esta publicación, incluido el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma, ni por ningún medio, sea este eléctrico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita del titular de los derechos.

Índice

Introducción	7
Objetivos	8
1 Una introducción a los modelos generativos	9
2 Modelos probabilísticos	11
2.1 Modelización probabilística	11
2.1.1 Dificultades	11
2.1.2 Representación	12
2.1.3 Inferencia	12
2.2 Revisión de conceptos de probabilidad	12
2.2.1 Elementos	12
2.2.2 Probabilidad condicional	13
2.2.3 Regla de la cadena	13
2.2.4 Independencia	13
2.3 Variables aleatorias	13
2.4 Funciones de distribución acumulativa	14
2.5 Funciones de masa de probabilidad	14
2.5.1 Funciones de densidad de probabilidad	15
2.5.2 Esperanza	15
2.5.3 Varianza	16
2.5.4 Algunas variables aleatorias comunes	17
2.5.5 Dos variables aleatorias	18
2.5.6 Teorema de Bayes	20
2.5.7 Independencia	21
2.5.8 Esperanza y covarianza	21
3 Modelos generativos basados en redes antagónicas o adversarias (GANs)	23
3.1 El concepto de redes antagónicas	24
3.2 Ventajas e inconvenientes de las GAN	25
3.3 Arquitecturas derivadas	26
3.3.1 GAN semisupervisada (SGAN)	26
3.3.2 Conditional GAN (CGAN)	26
3.3.3 Red antagónica generativa convolucional profunda (DCGAN)	27
3.3.4 Progressive GAN (PROGAN)	27
3.3.5 Self-attention GAN (SAGAN)	27
3.3.6 BigGAN	29
3.3.7 StyleGAN	29

3.4	f-GAN, una generalización de las GANs	31
3.4.1	Introducción	31
3.4.2	Estimación de modelos probabilísticos	31
3.4.3	GANs como modelos probabilísticos	32
3.4.4	Distancia entre distribuciones de probabilidad	32
3.4.5	Conclusión	35
3.4.6	Funciones de optimización mejoradas	35
3.5	Una aplicación de las GAN: aumento de datos	38
3.6	Conclusiones	39
4	Autocodificadores	40
4.1	Introducción a la estructura general de los autocodificadores .	40
4.2	Definición formal	41
4.3	Entrenamiento	41
4.4	Autocodificadores regularizados	41
4.4.1	Autocodificadores poco densos (SAE)	42
4.4.2	Autocodificadores de atenuación de ruido (DAE)	42
4.4.3	Autocodificador contractivo	43
4.5	Aplicaciones típicas	43
5	Modelos generativos basados en autocodificadores variacionales (VAEs)	45
5.1	Modelos probabilísticos	46
5.2	Optimización de modelos probabilísticos	46
5.3	Inferencia variacional (VI)	47
5.3.1	Variables observables vs. ocultas	47
5.3.2	Formalización matemática	47
5.3.3	Límite inferior de evidencia (ELBO)	48
5.3.4	ELBO como medio de optimizar q_ϕ	49
5.4	El codificador como forma de aproximar al posterior	50
5.5	Optimización del ELBO mediante gradiente estocástico	51
5.6	Cálculo de q_ϕ	53
5.7	Posteriores gaussianos	53
5.8	Posterior gaussiano con matriz de covarianza general	54
5.9	Desafíos de la arquitectura	55
5.10	Posteriores no gaussianos	56
5.11	Propuestas de extensión del autocodificador variacional original	56
5.12	Conclusiones	58
6	Modelos generativos basados en difusión	60
6.1	Formulación matemática	60
6.2	Modelos probabilísticos de eliminación del ruido producido por difusión	62
6.2.1	Proceso directo de difusión	62
6.2.2	Proceso de difusión inverso o de reconstrucción	63
6.2.3	Función de pérdida	64

6.2.4	Proceso de muestreo y entrenamiento	67
6.2.5	Implementación	67
6.2.6	Parametrización de β_t y Σ_θ	68
6.2.7	Conexión con los VAEs	68
6.2.8	Resumen	69
6.3	Generación basada en puntuación mediante el uso de ecuaciones diferenciales	69
6.3.1	Recordatorio de ecuaciones diferenciales	69
6.3.2	El proceso de difusión directa como una SDE	70
6.3.3	El proceso de difusión inverso mediante SDEs	71
6.3.4	Función de puntuación	71
6.4	Conclusiones	72
Resumen		74
Glosario		75

Introducción

En este módulo se introducen los modelos generativos basados en arquitecturas de aprendizaje profundo. La finalidad de estos modelos es la generación de nuevos datos que se asemejan a los de entrenamiento. Veremos distintas formas de aproximar el problema, todas ellas basadas en aprendizaje profundo, entre las que se encuentran las redes de generación adversarias, los autocodificadores variacionales (y arquitecturas derivadas) y los modelos basados en los mecanismos de difusión.

El objetivo de este módulo es presentar los conocimientos de base necesarios para introducirse en el ámbito de los modelos generativos y estar en condiciones de profundizar en estos aspectos mediante la consulta de la bibliografía especializada. Este campo de conocimiento está en profunda evolución, por ello es clave no entender los conocimientos aquí presentados como un tratado concluso, sino como una puerta desde la que acceder a un campo de conocimiento que va mucho más allá de este módulo.

Objetivos

Los objetivos que se pretende conseguir en este módulo se pueden resumir en los siguientes puntos:

- 1.** Definir una nueva clase de modelos predictivos, denominados generativos.
- 2.** Presentar algunas de las propuestas existentes para abordar el problema de generación de datos.
- 3.** Introducir los fundamentos teóricos de las tres propuestas más exitosas hasta el momento para abordar el problema; a saber, las redes de generación antagónicas adversarias, autocodificadores variacionales y los modelos basados en mecanismos de difusión.

1. Una introducción a los modelos generativos

En los módulos anteriores hemos estudiado diversas arquitecturas de redes neuronales que permiten modelizar funciones complejas para la evaluación de variables, todas ellas con finalidades de clasificación o de regresión. Estos modelos quedan recogidos bajo la clase de los denominados **modelos discriminativos**.

En este módulo didáctico introduciremos una clase distinta de modelos: los **modelos generativos**. A diferencia de los primeros, su finalidad no estriba en la discriminación de los datos de entrada, sino en la generación de instancias de datos equivalentes.

Formalmente, dado un conjunto de datos $\mathbf{X}$ y unas etiquetas asociadas a los mismos $\mathbf{y}$, el objetivo de los modelos discriminativos sería predecir funciones del tipo $P(\mathbf{y}|\mathbf{X})$, mientras que el de los generativos sería capturar $P(\mathbf{X},\mathbf{y})$ o simplemente $p(\mathbf{x})$ de manera que sea posible generar $\mathbf{x} \in \mathbf{X}$ o incluso pares $\mathbf{x},\mathbf{y} \in \mathbf{X}$.

El objetivo de los modelos generativos es, por tanto, la generación de datos similares (idealmente pertenecientes) a la población de la que deriva la muestra de datos que conforma el conjunto de entrenamiento.

La tarea que se ha de abordar es significativamente más compleja que la que resuelven los modelos discriminativos. Por ejemplo, un modelo generativo de imágenes debe ser capaz de capturar las correlaciones existentes entre los diferentes elementos que aparecen en la imagen, de forma que muestre de manera conjunta aquellos que es más probable que aparezcan juntos. Un modo de conseguir esto es capturando sus elementos constitutivos. Eso requiere, idealmente, ser capaz de identificar la existencia de esos elementos, así como su modelización independiente. Las distribuciones asociadas suelen ser complejas, en contraste con las regularidades estadísticas que es necesario distinguir para establecer una clasificación típica de un modelo discriminativo.

Esta diferenciación que hemos establecido también la podríamos analizar desde otra perspectiva, a saber, el tipo de datos que son necesarios para resolver estos problemas. Los modelos discriminativos históricamente se han resuelto utilizando un enfoque *supervisado*, es decir, los datos están formados por un conjunto de instancias y un conjunto de etiquetas que han sido previamente identificadas por humanos expertos. En los modelos generativos no es necesaria la existencia de etiquetas, pues el objetivo de los mismos es la inferencia

de propiedades internas a partir, únicamente, de la observación del conjunto de entrenamiento. El objetivo posterior es la generación de nuevas muestras que copien las propiedades definitorias de la población a la que representan. El tipo de aprendizaje requerido en este tipo de modelos es *no supervisado*.

El desafío conceptual existente consiste en capturar la distribución de probabilidad de la población representada por los datos de entrada (Datos de entrenamiento $\sim P_{\text{datos}}(x)$), en un modelo que permita generar nuevas instancias que emulen el comportamiento observado (Muestras generadas $\sim P_{\text{modelo}}(z)$).

No únicamente nos interesa conseguir este objetivo, sino también hacerlo de un modo similar al que lo hacemos los seres inteligentes, esto es, comprimiendo la información en entidades abstractas útiles. Una posible forma de abordar el problema es definir lo que denominamos *variables latentes*, de menor dimensión representativa que la real, y buscar un modelo algorítmico que permita encontrar la función capaz de reducir las dimensiones, maximizando la retención de información útil. Si el modelo resultante permite comprimir la información en un conjunto de variables reducido, se puede esperar que dichas variables contengan la información esencial y distintiva de los datos fuente, pudiéndose utilizar esa información para controlar la generación de datos en base al valor deseado de las variables latentes. Esta diferenciación en variables definitorias puede ser usada, por ejemplo, para el ajuste y la eliminación de sesgos en el conjunto de datos original, aumentando artificialmente aquellas variables separadas menos representadas. El conocimiento de estas variables también puede ser útil para la identificación de valores raros, es decir, *outliers*. Dependiendo del objetivo que persiga nuestro modelo, nos servirá para identificarlos y separarlos (por ejemplo, en sistemas de control de calidad de producto), o bien para incorporarlos como situaciones excepcionales a nuestro modelo y mejorar así su capacidad predictiva (por ejemplo, situaciones excepcionales encontradas en sistemas de conducción asistida).

Como hemos descrito, los modelos generativos son, en esencia, más complejos que los discriminativos y tienen un ámbito de aplicabilidad muy extenso. En los próximos apartados introduciremos varias formas de abordar este desafío predictivo. Estudiaremos los autocodificadores, las redes generativas adversarias y los modelos basados en difusión, todos ellos para la generación de imagen. Los modelos generativos de texto se estudian en el módulo dedicado a los transformadores.

2. Modelos probabilísticos

En esta introducción vamos a explicar los conceptos básicos de probabilidad (Koller y Friedman, 2009) que son claves para comprender la naturaleza probabilística de la mayoría de los modelos generativos existentes, especialmente los autocodificadores variacionales.

2.1 Modelización probabilística

Cuando tratamos de predecir eventos del mundo real utilizando las matemáticas, solemos usar operaciones entre variables que en su conjunto definen lo que denominamos modelos matemáticos. A veces puede interesar relacionar variables probabilísticas, esto es, variables que no tienen un valor numérico definido, sino que su valor está sometido a la incertidumbre. En estos casos, se pueden modelizar sus relaciones mediante distribuciones de probabilidad.

2.1.1 Dificultades

La modelización probabilística tiene una serie de dificultades que vamos a intentar presentar a través de un ejemplo:

Supongamos que disponemos de un modelo probabilístico paramétrico para realizar la clasificación de correos de *spam*. Este modelo podría tener la forma $p_{\theta}(y, x_1, \dots, x_n)$, donde y es una variable binaria que puede tomar los valores *spam* o *no spam* y $x_1, x_2, \dots, x_n$ las variables dependientes que supondremos, a efectos de simplificación, todas binarias. Si quisiéramos clasificar un nuevo correo nos fijaríamos en el valor de la función $p(y = 1 | x_1, \dots, x_n)$, esto es, la función que responde a la pregunta de cuál es la probabilidad de que y sea *spam* condicionado a que $x_1, \dots, x_n$ tengan un valor determinado.

Si nos preguntamos sobre el dominio de la función que acabamos de definir nos daremos cuenta de que es inmenso. Si tuviéramos que expresar en forma de tabla todos los valores que puede tomar la función, necesitaríamos un total de 2^{n+1} entradas. Si n fuera igual al tamaño del vocabulario nos daríamos cuenta de que sería claramente intratable, tanto desde el punto de vista computacional como estadístico. Los métodos probabilísticos suelen tener relaciones entre variables que escalan de forma exponencial. Una manera de convertirlos en tratables es mediante la simplificación en base a la realización de suposiciones.

2.1.2 Representación

La representación no es un problema trivial. Ya hemos visto en el ejemplo del problema del *spam*, de forma general, que para un mensaje que puede contener n palabras distintas, debemos especificar en el caso más general y sin suposiciones, $\mathcal{O}(2^n)$ parámetros.

2.1.3 Inferencia

Supongamos que hemos sido capaces de encontrar la distribución conjunta de probabilidad de las variables que lo integran, $p(x_1, x_2, \dots, x_n)$. A continuación enumeramos las preguntas típicas que estaríamos interesados en responder.

- *Inferencia marginal*: evaluación de la probabilidad de que una variable x_1 tome un valor independientemente del resto. $p(x_1) = \sum_{x_2} \sum_{x_3} \dots \sum_{x_n} p(x_1, x_2, \dots, x_n)$
- *Inferencia Maximum a posteriori (MAP)*: da como resultado cuál es la configuración de variables más probable $\operatorname{argmax}_{x_1, \dots, x_n} p(x_1, \dots, x_n)$.

En general, la inferencia sobre la distribución de probabilidad suele requerir operaciones de complejidad NP-hard. La dificultad de estas operaciones está relacionada también con la estructura del grafo que describe la probabilidad, esto es, en el fondo con las características de la función de probabilidad.

2.2 Revisión de conceptos de probabilidad

2.2.1 Elementos

A continuación realizamos la definición de algunos elementos básicos necesarios para la formalización de la probabilidad.

- **Espacio de la muestra Ω** : es el conjunto de todos los resultados posibles de un experimento aleatorio. Cada suceso $\omega \in \Omega$ se puede ver como una descripción completa de un estado en el mundo real al final del experimento.
- **Conjunto de eventos (o espacio de eventos)**: un conjunto cuyos elementos $A \in \mathcal{F}$ (llamados eventos) son un subconjunto de Ω (i.e. $(A \subseteq \Omega$ es un subconjunto de los posibles resultados de un experimento)).
- **Medida de probabilidad**: una función $P : \mathcal{F} \rightarrow \mathbb{R}$ con las siguientes propiedades:
 - $P(A) \geq 0$, para todo $A \in \mathcal{F}$

Nota

Este subapartado está adaptado de Maleki y Do (2000).

- Si $A_1, A_2, \dots$ son eventos disjuntos (esto es, $A_i \cap A_j = \emptyset$ cuando $i \neq j$), entonces

$$P(\cup_i A_i) = \sum_i P(A_i)$$
- $P(\Omega) = 1$

A estas tres **propiedades** se las denomina **axiomas de probabilidad**.

2.2.2 Probabilidad condicional

Sea B un suceso con probabilidad distinta de cero. Se define la probabilidad condicionada de un evento A habiendo sucedido B como:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (1)$$

En otras palabras, $P(A|B)$ es la probabilidad de que suceda A después de haber observado que ha sucedido el evento B .

2.2.3 Regla de la cadena

Sea $S_1, \dots, S_k$ una serie de eventos donde $P(S_i) > 0$. La regla de la cadena dice que:

$$P(S_1 \cap S_2 \cap \dots \cap S_k) = P(S_1)P(S_2|S_1)P(S_3|S_2 \cap S_1) \dots P(S_k|S_1 \cap S_2 \cap \dots \cap S_{k-1}) \quad (2)$$

En el caso de $k = 2$, entonces la regla de la cadena coincide con la ecuación 1.

2.2.4 Independencia

Se dice que dos eventos son independientes si $P(A \cap B) = P(A)P(B)$, o bien equivalentemente cuando $P(A|B) = P(A)$. Intuitivamente, A y B son independientes cuando conocer B no afecta a la probabilidad de A .

2.3 Variables aleatorias

Una variable aleatoria X es una función $X : \Omega \rightarrow E$, donde E es un espacio medible. Habitualmente, se representa con mayúsculas las variables aleatorias, y con minúsculas los valores que puede tomar. Así, $X = x$ significa que asignamos el valor $x \in E$ a la variable aleatoria X . Para variables reales representamos como $P(a \leq X \leq b) := P(\{\omega : a \leq X(\omega) \leq b\})$.

Si queremos indicar que la variable A toma el valor 1, lo podemos hacer de la siguiente forma $\mathbf{1}\{A\}$. Otro ejemplo sería:

$$\mathbf{1}\{X > 3\} = \begin{cases} 1 & \text{si } x > 3 \\ 0 & \text{si } x \leq 3 \end{cases} \quad (3)$$

2.4 Funciones de distribución acumulativa

Una distribución acumulativa de probabilidad (CDF) es una función $F_X(x) : \mathbb{R} \rightarrow [0,1]$ que mide la probabilidad como $F_X(x) = P(X \leq x)$.

Propiedades:

- $0 \leq F_X(x) \leq 1$
- $\lim_{x \rightarrow -\infty} F_X(x) = 0$
- $\lim_{x \rightarrow +\infty} F_X(x) = 1$
- $x \leq y \Rightarrow F_X(x) \leq F_X(y)$

2.5 Funciones de masa de probabilidad

En el caso en el que el número de posibles valores que puede tomar una variable aleatoria X sea finito, una forma más sencilla de representar la probabilidad es especificar directamente su valor para cada valor que pueda tomar. La función de masa de probabilidad (PMF) es una función $p_X : \mathbb{R} \rightarrow [0,1]$ para la que $p_X(x) = P(X = x)$. Para el caso de una variable aleatoria discreta, utilizamos la notación $Val(X)$ para referirnos al conjunto de todos los posibles valores que X puede tener. Por ejemplo, en el caso de que $X(\omega)$ fuera una variable aleatoria referida al número de caras que pueden salir al tirar 10 veces seguidas una moneda al aire, entonces $Val(X) = \{0,1,2,\dots,10\}$.

Propiedades:

- $0 \leq p_X(x) \leq 1$
- $\sum_{x \in Val(X)} p_X(x) = 1$
- $\sum_{x \in A} p_X(x) = P(X \in A)$

2.5.1 Funciones de densidad de probabilidad

Para algunas variables aleatorias continuas, la función distributiva acumulativa es diferenciable en todo su dominio. En estos casos, se puede definir la función de densidad de probabilidad (PDF) como la derivada de la CDF, a saber:

$$f_X(x) = \frac{dF_X(x)}{dx}$$

Cabe puntualizar que la PDF de una variable aleatoria continua no siempre existe (por ejemplo, en aquellos casos donde no es diferenciable en todos sus puntos).

De acuerdo con las propiedades de derivación, para un pequeño δx , $P(x \leq X \leq x + \delta x) \approx f_X(x)\delta x$.

Tanto los CDFs como los PDFs (cuando existen) se pueden usar para calcular probabilidades. Es importante hacer notar que $f_X(x) \neq P(X = x)$.

Propiedades:

- $f_X(x) \geq 0$
- $\int_{-\infty}^{+\infty} f_X(x) dx = 1$
- $\int_{x \in A} f_X(x) dx = P(X \in A)$

2.5.2 Esperanza

Supongamos que X es una variable aleatoria discreta con una PMF $p_X(x)$ y $g : \mathbb{R} \rightarrow \mathbb{R}$ una función arbitraria. En este caso, $g(X)$ puede ser considerada también una variable aleatoria. Definimos la **esperanza** o **valor esperado** de $g(X)$ como

$$\mathbb{E}[g(X)] = \sum_{x \in \text{Val}(X)} g(x)(p_X(x))$$

Si X es una variable aleatoria continua con una PDF $f_X(x)$, entonces el valor esperado de $g(X)$ se define como

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x)f_X(x) dx$$

Intuitivamente, la esperanza de $g(X)$ puede ser vista como una media ponderada de los valores de $g(x)$ sobre todos los valores de x , donde los pesos vienen dados por $p_X(x)$ o $f_X(x)$. Como caso particular, cabe observar que $\mathbb{E}[X]$ se obtiene cuando $g(x) = x$; este valor es conocido también como la media de la variable X .

Propiedades:

- $\mathbb{E}[a] = a$ para cualquier constante $a \in \mathbb{R}$
- $\mathbb{E}[af(X)] = a\mathbb{E}[f(X)]$ para cualquier constante $a \in \mathbb{R}$
- (Linealidad de la esperanza) $\mathbb{E}[f(X) + g(X)] = \mathbb{E}[f(X)] + \mathbb{E}[g(X)]$
- Para una variable discreta aleatoria X , $\mathbb{E}[\mathbf{1}\{X = k\}] = P(X = k)$

2.5.3 Varianza

La varianza de una variable aleatoria X es una medida de lo concentrada que se encuentra la variable X alrededor de su media. Formalmente, la variancia se define como $\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$

Utilizando la propiedad de la esperanza definida en el subapartado anterior, podemos derivar definiciones alternativas.

$$\begin{aligned}
 \mathbb{E}[(X - \mathbb{E}[X])^2] &= \\
 &= \mathbb{E}[X^2 - 2\mathbb{E}[X]X + \mathbb{E}[X]^2] = \\
 &= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 = \\
 &= \mathbb{E}[X^2] - \mathbb{E}[X]^2
 \end{aligned}$$

Propiedades:

- $\text{Var}[a] = 0$ para cualquier constante $a \in \mathbb{R}$
- $\text{Var}[af(X)] = a^2 \text{Var}[f(X)]$ para cualquier constante $a \in \mathbb{R}$

2.5.4 Algunas variables aleatorias comunes

Variables aleatorias discretas

- $X \sim \text{Bernoulli}(p)$ donde $(0 \leq p \leq 1)$: por ejemplo, el resultado de tirar una moneda al aire para la que la probabilidad de salir cara es p .

$$p(\mathbf{x}) = \begin{cases} p & \text{si } x = 1 \\ 1-p & \text{si } x = 0 \end{cases}$$

- $X \sim \text{Binomial}(n, p)$ donde $(0 \leq p \leq 1)$: por ejemplo, la probabilidad de obtener x caras en n tiradas de moneda independientes, donde p es la probabilidad de que salga cara en una tirada.

$$p(\mathbf{x}) = \binom{n}{x} \cdot p^x (1-p)^{n-x}$$

- $X \sim \text{Geométrica}(p)$ donde $(0 \leq p \leq 1)$: por ejemplo, la probabilidad de tener que realizar x tiradas independientes para obtener la primera cara, donde p es la probabilidad de que salga cara en una tirada.

$$p(\mathbf{x}) = p(1-p)^{(x-1)}$$

- $X \sim \text{Poisson}(\lambda)$ donde $(\lambda > 0)$: distribución de probabilidad sobre los enteros positivos para modelizar la frecuencia de eventos raros.

$$p(\mathbf{x}) = e^{-\lambda} \frac{\lambda^x}{x!}$$

Variables aleatorias continuas

- $X \sim \text{Uniforme}(a, b)$ donde $(a < b)$: densidad de probabilidad igual para cualquier valor situado entre a y b .

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{si } a \leq x \leq b \\ 0 & \text{otros} \end{cases}$$

- $X \sim \text{Exponencial}(\lambda)$ donde $(\lambda > 0)$: densidad de probabilidad de decaimiento sobre los reales no negativos.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{si } x \geq 0 \\ 0 & \text{otros} \end{cases}$$

- $X \sim \text{Normal}(\mu, \sigma^2)$ conocida como distribución gaussiana.

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

2.5.5 Dos variables aleatorias

Hasta ahora hemos considerado eventos con una única variable aleatoria. Sin embargo, es habitual encontrarse con experimentos donde hay más de una variable. Por ejemplo, en el experimento de lanzar una moneda al aire podemos considerar, por una parte, el número total de caras que ha salido y, por otro, el número máximo de caras consecutivas que ha salido, que serían dos variables aleatorias.

Distribuciones conjunta y marginal

Supongamos que tenemos dos variables aleatorias X e Y . Una forma de tratar con estas variables es hacerlo por separado. Si lo hacemos de este modo únicamente necesitaríamos $F_X(x)$ y $F_Y(y)$. Pero si queremos saber acerca de la interacción simultánea de las dos variables en un experimento aleatorio, entonces necesitamos una estructura más complicada llamada distribución acumulativa de X e Y , definida como:

$$F_{XY}(x, y) = P(X \leq x, Y \leq y)$$

Conociendo la distribución acumulativa de X e Y se puede conocer la probabilidad de cualquier evento que incluya a X e Y .

La CDF conjunta $F_{XY}(x, y)$ y las distribuciones acumulativas $F_X(x)$ y $F_Y(y)$ están relacionadas del siguiente modo:

$$F_X(x) = \lim_{y \rightarrow \infty} F_{XY}(x, y)$$

$$F_Y(y) = \lim_{x \rightarrow \infty} F_{XY}(x, y)$$

A $F_X(x)$ y $F_Y(y)$ se les denomina distribuciones acumulativas marginales.

Propiedades:

- $0 \leq F_{XY}(x,y) \leq 1$
- $\lim_{x,y \rightarrow \infty} F_{XY}(x,y) = 1$
- $\lim_{x,y \rightarrow -\infty} F_{XY}(x,y) = 0$
- $F_X(x) = \lim_{y \rightarrow \infty} F_{XY}(x,y)$

Funciones de masa de probabilidad conjunta y marginal

Sean X e Y variables aleatorias discretas. Entonces la función conjunta de masa de probabilidad $p_{XY} : \text{Val}(X) \times \text{Val}(Y) \rightarrow [0,1]$ se define como:

$$p_{XY}(x,y) = P(X = x, Y = y)$$

Aquí, $0 \leq p_{XY}(x,y) \leq 1$ para todo x,y y $\sum_{x \in \text{Val}(X)} \sum_{y \in \text{Val}(Y)} p_{XY}(x,y) = 1$

Podemos relacionar también $P_X(x)$ con $p_{XY}(x,y)$ del siguiente modo:

$$p_X(x) = \sum_y p_{XY}(x,y)$$

Nos referimos a $p_X(x)$ como la función de masa probabilística marginal. El proceso de anular el efecto de una de las variables se denomina marginalización.

Funciones de densidad de probabilidad conjunta y marginal

Sean X e Y dos funciones continuas aleatorias con distribución conjunta de probabilidad F_{XY} . En caso de que $F_{XY}(x,y)$ sea diferenciable en todo su dominio podemos definir la función de densidad de probabilidad conjunta como:

$$f_{XY}(x,y) = \frac{\partial^2 F_{XY}(x,y)}{\partial x \partial y}$$

Como en el caso unidimensional, $f_{XY}(x,y) \neq P(X = x, Y = y)$, pero

$$\int \int_{(x,y) \in A} f_{XY}(x,y) dx dy = P((X,Y) \in A)$$

donde $\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f_{XY}(x,y) = 1$

Análogamente al caso discreto, definimos

$$f_X(x) = \int_{-\infty}^{+\infty} f_{XY}(x,y) dy$$

como la función de densidad de probabilidad marginal (o densidad marginal) de X . De forma análoga con $f_Y(y)$.

Distribuciones condicionales

El objetivo de las distribuciones condicionales de probabilidad es la integración de la influencia que ejerce de la información de sucesos relacionados en la probabilidad de un evento. Suponiendo que $P_X(x) \neq 0$, entonces

$$p_{Y|X}(y|x) = \frac{p_{XY}(x,y)}{p_X(x)}$$

En el caso continuo y por analogía con el caso discreto, suponiendo que $f_X(x) \neq 0$,

$$f_{Y|X}(y|x) = \frac{f_{XY}(x,y)}{f_X(x)}$$

2.5.6 Teorema de Bayes

El teorema de Bayes se puede representar con la ecuación 4.

$$P_{Y|X}(y|x) = \frac{P_{XY}(x,y)}{P_X(x)} = \frac{P_{XY}(x|y)P_Y(y)}{\sum_{y' \in \text{Val}(Y)} P_{X|Y}(x|y')P_Y(y')} \quad (4)$$

A partir de esta ecuación se deriva una terminología que conviene conocer:

- **Prior**, $P_Y(y)$: probabilidad de que suceda y antes de conocer x .
- **Posterior**, $P_{Y|X}(y|x)$, probabilidad de que suceda y conociendo que ha sucedido x .
- **Evidence**, $P_X(x)$ probabilidad del evento contra el que queremos condicionar.
- **Likelihood**, $P_{XY}(x|y)$

En el caso de que X e Y sean variables continuas:

$$f_{Y|X}(y|x) = \frac{f_{XY}(x,y)}{f_X(x)} = \frac{f_{XY}(x|y)f_Y(y)}{\int_{-\infty}^{+\infty} f_{X|Y}(x|y')f_Y(y') dy'} \quad (5)$$

2.5.7 Independencia

Dos variables aleatorias X e Y son independientes si $F_{XY}(x,y) = F_X(x) \cdot F_Y(y)$.

Equivalentemente:

- Para variables aleatorias discretas, $p_{XY}(x,y) = p_X(x) \cdot p_Y(y)$ para todo $x \in \text{Val}(X)$ y $y \in \text{Val}(Y)$.
- Para variables aleatorias discretas, $p_{Y|X}(y|x) = p_Y(y)$ siempre que $p_X(x) \neq 0$ para todo $y \in \text{Val}(Y)$.
- Para variables aleatorias continuas, $f_{XY}(x,y) = f_X(x)f_Y(y)$ para todo $x,y \in \mathbb{R}$.
- Para variables aleatorias continuas, $f_{Y|X}(y|x) = f_Y(y)$ siempre que $f_X(x) \neq 0$ para todo $y \in \mathbb{R}$.

2.5.8 Esperanza y covarianza

Supongamos que tenemos dos variables aleatorias X,Y y $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ es una función de dichas variables. Entonces, el valor esperado de g se define como:

$$\mathbb{E}[g(X,Y)] = \sum_{x \in \text{Val}(X)} \sum_{y \in \text{Val}(Y)} g(x,y) p_{XY}(x,y)$$

Para el caso de variables continuas, la expresión sería la siguiente:

$$\mathbb{E}[g(X,Y)] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x,y) f_{XY}(x,y) dx dy$$

Se define la covarianza como:

$$\text{Cov}[X,Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$$

Propiedades:

- Linealidad de la esperanza: $\mathbb{E}[f(X,Y) + g(X,Y)] = \mathbb{E}[f(X,Y)] + \mathbb{E}[g(X,Y)]$
- $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] + 2\text{Cov}[X,Y]$

- Si X e Y son independientes, entonces $\text{Cov}[X, Y] = 0$
- Si X e Y son independientes, entonces $\mathbb{E}[f(X)g(Y)] = \mathbb{E}[f(X)]\mathbb{E}[g(Y)]$

Estas ecuaciones son fácilmente generalizables para el caso de variables múltiples. En la bibliografía se pueden encontrar referencias donde consultar las ecuaciones para esos casos.

3. Modelos generativos basados en redes antagónicas o adversarias (GANs)

Las redes antagónicas generativas o redes adversarias generativas (GANs) [Creswell y otros (2018), Goodfellow y otros (2020), Cohen y Giryes (2022)] son un método para la optimización competitiva entre dos redes neuronales, una llamada generadora y otra discriminadora, con el objetivo de conseguir generar nuevas instancias idealmente indistinguibles a las pertenecientes a la distribución de probabilidad de la que derivan los datos de entrenamiento.

El fundamento teórico general del que derivan permite su utilización para la generación de cualquier tipo de datos, habiéndose demostrado efectiva en campos diversos como son la visión por computador [Dziugaite y otros (2015), Karras y otros (2017), Ledig y otros (2017)], la segmentación semántica [Luc y otros (2016), Isola y otros (2017), Wang y otros (2018), Hoffman y otros (2018)], la síntesis de series temporales (Hartmann y otros (2018)), la edición de imagen [Shaham y otros (2019), Lample y otros (2017), Abdal y otros (2021), Xia y otros (2022)], el procesamiento del lenguaje natural [Féds y otros (2018), Jetchev y otros (2016), Guo y otros (2018)], la generación de imagen a partir de texto [Ramesh y otros (2021), Radford y otros (2021), Patashnik y otros (2021)] entre otros.

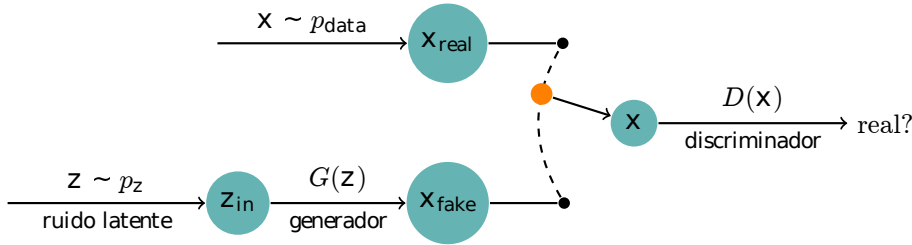
Para cualquier conjunto de datos, podemos hipotetizar que es posible definir una distribución de probabilidad p_{data} representativa de la población representada por la muestra formada por el conjunto de datos. De ser esto posible, para cualquier valor de $\mathbf{x}$ será posible establecer un valor $P_{data}(\mathbf{x})$ que determine la probabilidad de que $\mathbf{x}$ pertenezca a la población. De existir una función de este tipo, sería una función discriminativa que dada una instancia permitiría conocer la probabilidad de pertenencia a la población. Los modelos generativos modelizan la distribución de probabilidad mencionada pero no proporcionan un valor de probabilidad, sino que generan instancias nuevas que pertenecen a distribuciones de probabilidad próximas a la que pretenden asemejar. Las GANs definen un esquema de aprendizaje que facilita la codificación de los atributos definitorios de la distribución de probabilidad en una red neuronal, de manera que la red incorpore la información esencial que le permite generar instancias pertenecientes a distribuciones de probabilidad próximas a la del conjunto de datos que pretende representar.

En el siguiente subapartado se presenta el esquema básico de la arquitectura GAN y su aspecto distintivo, la naturaleza de la función objetivo utilizada para su optimización. Posteriormente, se presentan las arquitecturas y funciones de objetivo derivadas, así como sus aplicaciones.

3.1 El concepto de redes antagónicas

La arquitectura GAN está formada por dos redes neuronales constituyentes: una denominada discriminadora (D) y otra generadora G . La red G se encarga de generar nuevas instancias del mismo dominio que el del conjunto de datos de origen. La red D se encarga de discriminar si los datos de entrada son reales, esto es, pertenecientes al conjunto de datos de entrada, o bien son ficticios, esto es, generados artificialmente. Ambas redes se entrenan conjuntamente, de manera que G maximice sus posibilidades de no ser detectada por D y D de forma que haga cada vez más sofisticados sus métodos de detección de los datos generados artificialmente por G . Estas dos redes adversarias compiten en un juego de suma cero en el que se hipotetiza que eventualmente llegan a un equilibrio de Nash (Moghadam y otros, 2021).

Figura 1. Diagrama representativo del proceso de entrenamiento de las redes adversarias generativas (GANs)



En la figura 1 se muestra un diagrama representativo del proceso de optimización de las GAN. Un vector $\mathbf{z}$ es un muestreo de una distribución de probabilidad aleatoria p_z , $\mathbf{z} \sim p_z$ y alimentado como entrada a G . El propósito de la optimización es conseguir que $G(\mathbf{z}) \sim p_g$ acabe siendo una estimación de la distribución de probabilidad P_{data} . Las GANs se optimizan mediante la función min-max de un juego de suma cero, expresado por la ecuación 6.

$$\min_G \max_D \mathbb{E}_{x \sim p_r} \log[D(\mathbf{x})] + \mathbb{E}_{z \sim p_z} \log[1 - D(G(\mathbf{z}))] \quad (6)$$

Equivalentemente, sean p_θ , D_ω las redes neuronales generadora y discriminadora de una GAN, siendo θ los parámetros de G y ω los de D . Ambas redes se optimizan en conjunto con la función objetivo definida por la ecuación 7.

$$\min_\theta \max_\omega \mathbb{E}_{x \sim Q} \log[D_\omega(x)] + \mathbb{E}_{x \sim p_\theta} \log[1 - D_\omega(x)] \quad (7)$$

Durante el proceso de optimización, la red D recibirá como entrada de manera aleatoria datos pertenecientes al conjunto de datos y otros procedentes de la red G . Se optimizará su funcionamiento para que su discriminación sea efectiva (ecuación 8).

$$\nabla_{\theta_D} \frac{1}{m} \sum_{i=1}^m [\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))] \quad (8)$$

Al mismo tiempo, cuando D reciba una entrada procedente de G , este se optimizará para mejorar sus predicciones y hacer cada vez más difícil el papel de D . Esto únicamente se puede conseguir mejorando la calidad de los datos generados y haciéndolos más parecidos al conjunto de datos original (ecuación 9).

$$\nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}^{(i)}))) \quad (9)$$

Se establece así una competición entre las dos redes (de ahí el nombre de adversarias), de forma que idealmente en el progreso de este proceso ambas mejoran su funcionamiento al punto que idealmente el generador acaba produciendo datos cada vez más parecidos a los del conjunto de datos original.

3.2 Ventajas e inconvenientes de las GAN

Desde su introducción en 2014, las GANs han despertado un gran interés sobre todo en el campo de la generación de imagen. Esto se ha debido a que presentan una serie de ventajas sobre el otro paradigma dominante hasta el momento en lo que a modelos generativos se refiere, los VAEs (Kingma y Welling, 2013). Dichas ventajas son las siguientes:

- **Imágenes más nítidas:** las GANs producen imágenes más nítidas que otros modelos generativos disponibles hasta el momento. Los modelos de difusión que veremos más adelante son una excepción posterior en este aspecto.
- **Tamaño configurable:** el tamaño de la variable aleatoria no está restringido, pudiéndose enriquecer en caso de ser necesario.
- **Generador versátil:** el paradigma de diseño basado en GANs soporta distintos tipos de funciones generadoras, a diferencia de otros modelos generativos que pueden tener restricciones debido a su arquitectura. Los VAEs, por ejemplo, obligan a utilizar una función gaussiana en la primera capa del generador.

La arquitectura también tiene sus desventajas, entre las que están las siguientes:

- **Colapso de modo:** durante el entrenamiento sincronizado de generador y discriminador, el generador puede tener tendencia a reproducir únicamente un modo específico que es capaz de burlar al discriminador. A pesar de que este patrón puede estar minimizando la función objetivo, lo hace sin cubrir todo el dominio del conjunto de datos.
- **Desvanecimiento de gradientes:** a veces el discriminador se optimiza demasiado rápido en su función. En estos casos, los gradientes que propaga pueden ser demasiado bajos para asegurar la optimización del generador.
- **Inestabilidad:** a menudo durante el entrenamiento los parámetros de ambas redes fluctúan sin encontrar un punto de equilibrio. En estas circunstancias el generador tiene dificultades en encontrar un punto que genere imágenes de alta calidad.

3.3 Arquitecturas derivadas

En este subapartado se presentan aquellas arquitecturas derivadas de la original que mejoran su rendimiento en alguna de las desventajas mencionadas.

3.3.1 GAN semisupervisada (SGAN)

SGAN (Odena, 2016) incluye una variación en el discriminador que le permite aprovechar las ventajas de contar con datos supervisados. Consiste en añadir un cabezal adicional para la predicción de la clase de pertenencia. En aquellos casos reales que se conoce dicha clase, se utiliza el cabezal softmax de predicción para optimizar el discriminador. En aquellos casos que no se conozca, se utiliza la optimización vía clasificación binaria típica de la GAN convencional. Los resultados demuestran que este tipo de entrenamiento mejora las capacidades de SGAN respecto a la GAN original.

3.3.2 Conditional GAN (CGAN)

CGAN (Mirza y Osindero, 2014) modifica el método original introduciendo una entrada adicional tanto en el generador como en el discriminador. Esta entrada adicional sirve como condicionante para ambas funciones. Esta nueva información y se fusiona en el generador con el muestreo de la variable aleatoria z para posteriormente generar las nuevas instancias. Lo mismo ocurre en el discriminador, donde y se integra con los datos x a analizar. La nueva función de optimización queda como indica la ecuación 10.

$$\min_G \max_D \mathbb{E}_{x \sim p_r} \log[D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{z \sim p_z} \log[1 - D(G(\mathbf{z}|\mathbf{y}))] \quad (10)$$

3.3.3 Red antagónica generativa convolucional profunda (DCGAN)

Las redes antagónicas generativas convolucionales profundas (DCGAN) fueron introducidas por primera vez en Radford y otros (2015) como método para la generación de imágenes. Utilizan convoluciones en el discriminador y convoluciones traspuestas (Dumoulin y Visin, 2016) en el generador. Además de mejoras en la resolución de las imágenes generadas, consiguen mejoras en la estabilidad del entrenamiento que en su estudio atribuyen a la introducción de las siguientes modificaciones:

- Sustitución de todas las capas de *pooling* de las dos redes. En el discriminador se utilizan núcleos con *stride* mayor que 1 y en el generador convoluciones traspuestas para aumentar el tamaño de la imagen.
- Uso de la normalización por lotes en las dos redes.
- En el discriminador se cambia la función de activación de ReLU a Leaky-ReLU (Maas y otros, 2013). En el generador se utilizan ReLU en todas las capas excepto en la última donde se usa la función de activación tangente hiperbólica ($\tanh$).

3.3.4 Progressive GAN (PROGAN)

En Karras y otros (2017) se introduce un método progresivo de entrenamiento y aumento de la resolución de las imágenes generadas que da muy buenos resultados. Muchas de las arquitecturas GAN más exitosas usan este método. En el trabajo citado se empieza entrenando una red generativa de 4x4 para ir añadiendo capas en el generador y discriminador conforme va avanzando el entrenamiento hasta conseguir resoluciones de salida de 1024x1024. Conforme se van añadiendo capas, todas las capas anteriores siguen estando sometidas a los cambios inherentes a la optimización de las redes.

3.3.5 Self-attention GAN (SAGAN)

Muchas de las implementaciones existentes hasta el momento fallan en la captura de patrones geométricos y estructurales de largo alcance. Se hipotetiza que la causa de esto se debe a la naturaleza convolucional de las arquitecturas generativas. Debido a ella, las dependencias son de corto alcance y requieren del paso a través de varias capas para ser resueltas. Existen diversas soluciones que se pueden aplicar para solucionar este aspecto. La primera sería aumentar el tamaño de las convoluciones con el aumento asociado de los requerimientos computacionales. Otra sería el incremento de la profundidad de las redes. Una tercera posibilidad, que es la propuesta por SAGAN (Zhang y otros, 2019),

es la utilización de mecanismos de autoatención en alguna de las capas de la red convolucional.

Los atributos de salida $\mathbf{x} \in \mathbb{R}^{C \times N}$ de una capa de la red neuronal se transforman en dos espacios $\mathbf{f}(\mathbf{x}) = \mathbf{W}_f \mathbf{x}$ y $\mathbf{g}(\mathbf{x}) = \mathbf{W}_g \mathbf{x}$ para posteriormente calcular la atención como indica la ecuación 11.

$$\beta_{j,i} = \frac{\exp(s_{i,j})}{\sum_{i=1}^N \exp(s_{i,j})} \quad \text{donde} \quad s_{i,j} = \mathbf{f}(\mathbf{x}_i)^T \mathbf{g}(\mathbf{x}_j) \quad (11)$$

$\beta_{j,i}$ indica la atención que está prestando a la región i cuando está generando la j . C es el número de canales y N el número de atributos de la anterior capa. La salida de la capa de atención $\mathbf{o} = (\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_N) \in \mathbb{R}^{C \times N}$ se puede expresar como indica la ecuación 12.

$$\mathbf{o}_j = \mathbf{v} \left(\sum_{i=1}^N \beta_{j,i} \mathbf{h}(\mathbf{x}_i) \right), \quad \mathbf{h}(\mathbf{x}_i) = \mathbf{W}_h \mathbf{x}_i, \quad \mathbf{v}(\mathbf{x}_i) = \mathbf{W}_v \mathbf{x}_i \quad (12)$$

Donde $\mathbf{W}_g \in \mathbb{R}^{\tilde{C} \times C}$, $\mathbf{W}_f \in \mathbb{R}^{\tilde{C} \times C}$, $\mathbf{W}_h \in \mathbb{R}^{\tilde{C} \times C}$ y $\mathbf{W}_v \in \mathbb{R}^{C \times \tilde{C}}$ son matrices optimizadas en tiempo de entrenamiento, implementadas como convoluciones 1x1. En el artículo se fija $\tilde{C} = C/8$ por ser más eficiente de cara a la computación y, según se indica, no afectar significativamente a los resultados.

El valor de autoatención considerado se escala y se suma al valor del atributo de entrada, $\mathbf{y}_i = \gamma \mathbf{o}_i + \mathbf{x}_i$, siendo γ un parámetro optimizable que se inicializa a cero. Esto tiene su lógica, pues al inicio del entrenamiento se puede esperar resolver las dependencias locales para que, una vez avanzada la optimización, se afine resolviendo las dependencias de mayor alcance. Este mecanismo de atención se aplica tanto al generador como al discriminador. Ambos son optimizados minimizando una versión modificada de la función de optimización original, que toma la forma de la ecuación 13. Esta ecuación es la versión de máximo-margen (Hinge Loss) típica para la optimización de SVMs (Crammer y Singer, 2001).

$$L_D = -\mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim P_{data}} [\min(0, -1 + D(\mathbf{x}, \mathbf{y}))] - \mathbb{E}_{\mathbf{z} \sim p_z, \mathbf{y} \sim P_{data}} [\min(0, -1 - D(G(\mathbf{z}), \mathbf{y}))] \\ L_G = -\mathbb{E}_{\mathbf{z} \sim p_z, \mathbf{y} \sim P_{data}} G(G(\mathbf{z}), \mathbf{y}) \quad (13)$$

Junto con su propuesta presentan también una metodología de entrenamiento para estabilizar el inherentemente inestable proceso de optimización de las GANs.

3.3.6 BigGAN

BigGAN (Brock y otros, 2018) es un tipo de GAN diseñada para la generación, mediante escalado, de imágenes de alta resolución. Incluye una serie de cambios incrementales respecto a las redes anteriormente mencionadas, así como también algunas innovaciones.

Entre las mejoras incrementales de relevancia encontramos las siguientes:

- 1) Propuesta de arquitectura basada en SAGAN utilizando normalización espectral (Miyato y otros, 2018) tanto para D como para G y utilizando TTUR (Heusel y otros, 2017).
- 2) Al igual que SAGAN, utiliza la función de pérdida Hinge como objetivo de optimización.
- 3) Utiliza normalización por lotes condicionada a la clase (CBN) (De Vries y otros, 2017) (mediante proyección lineal) para proveer de información de la clase a G .
- 4) Utiliza un discriminador por proyección (Miyato y Koyama, 2018) para incorporar la información de la clase.

En cuanto a las innovaciones, cabe resaltar:

- 1) Incremento del tamaño de los lotes.
- 2) Incremento del tamaño de capa.
- 3) Adición de conexiones directas entre la variable latente z y las capas intermedias de la red.
- 4) Uso de una variante de regularización ortogonal (Brock y otros, 2016).

BigGAN consigue una mejora sustancial de la calidad de las imágenes generadas para tamaños de 128x128, 256x256 y 512x512, aumentando el número de parámetros (x4) e incrementando el tamaño del batch de entrenamiento (x8) respecto a SAGAN. Además de las modificaciones indicadas, introducen algunos cambios sobre las variables latentes durante el proceso de inferencia, consistentes en truncar los valores fuera de un rango determinado.

3.3.7 StyleGAN

StyleGAN (Karras y otros, 2019) propone una nueva arquitectura para el generador, manteniendo el mismo diseño para el discriminador. En la figura 2 se muestra un esquema de la arquitectura propuesta para la red generadora.

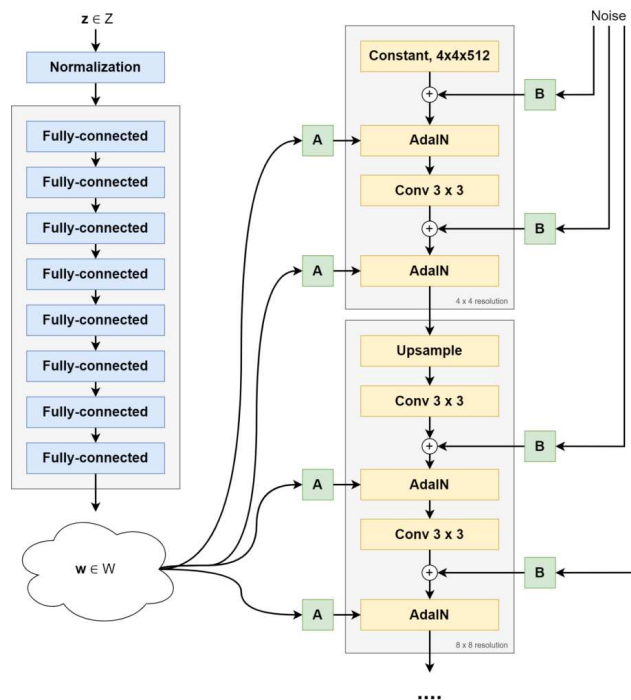
Two Time-scale Update Rule (TTUR)

Es una regla de actualización de los pesos de las GANs utilizando descenso de gradiente, consistente en el uso de tasas de aprendizaje distintas en generador y discriminador. Se sabe que el discriminador es capaz de converger a un mínimo local cuando el generador permanece estable. De cara a asegurar la convergencia, la idea es utilizar tasas de aprendizaje más bajas en el generador que en el discriminador. De esta manera aseguramos la convergencia de ambas redes a un punto de equilibrio de Nash.

Conditional Batch Normalization (CBN)

Es una variante de la normalización batch condicionada a la clase. Dado un mapa de atributos con C canales, donde γ_c y β_c representan los parámetros de normalización de cada canal y para los que la regla de actualización viene representada por $\hat{\beta}_c = \beta_c + \Delta\beta_c$ y $\hat{\gamma}_c = \gamma_c + \Delta\gamma_c$, entonces $\Delta\beta = MLP_1(e_q)$ y $\Delta\gamma = MLP_2(e_q)$. Para el caso de BigGAN MLP es una transformación lineal y e_q la clase.

Figura 2. Diagrama de la arquitectura StyleGAN



Fuente: <https://github.com/christianversloot/machine-learning-articles/>

StyleGAN tiene tres componentes característicos:

- 1) Crecimiento progresivo del tamaño de la imagen generada al estilo de PRO-GAN (bloques en amarillo) ($4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128 \rightarrow 256 \times 256 \rightarrow 512 \times 512 \rightarrow 1024 \times 1024$)
- 2) En los bloques de Upsample utiliza muestreo bilinear en vez de la copia del valor de los vecinos cercanos.
- 3) Sustitución del vector de entrada $z \sim P_z$ por una matriz de entrada constante de $4 \times 4 \times 512$, y se introduce una red de ruido gaussiano, que alimenta independientemente a cada una de las capas intermedias.
- 4) Introducción de la denominada *red de mapeo del ruido*, que, al paso por varias capas completamente conectadas, transforma una entrada aleatoria en una representación interna de los estilos. La salida de esta red actúa como entrada en los bloques AdaIN de cada capa, fijando los parámetros de sesgo y escalado del bloque con la finalidad de actuar como estilos adaptativos.
- 5) AdaIN. Normalización adaptativa de las instancias: introducida inicialmente en Huang y Belongie (2017). StyleGAN la utiliza tanto como capa de normalización como para establecer el estilo a través de los parámetros de escalado y sesgo que son derivados de las representaciones internas aprendidas por la red de mapeo de ruido.

3.4 f-GAN, una generalización de las GANs

En los subapartados previos, hemos presentado a las GANs como una herramienta para generar datos, que se optimiza para producir resultados que se parezcan a los datos de entrenamiento. Se ha explicado cómo este proceso se lleva a cabo a través de la optimización de un modelo parametrizable en forma de red neuronal, midiendo la diferencia entre la distribución del modelo y la real durante el proceso de entrenamiento.

En este subapartado, exploraremos cómo distintas formas de medir esta distancia pueden dar lugar a diferentes funciones objetivo y, por lo tanto, a distintas redes resultantes. Este concepto se ha formalizado en el artículo de Nowozin y otros (2016), que proporciona una generalización del concepto de distancia en GANs para diferentes formas de medir la distancia entre distribuciones de probabilidad.

3.4.1 Introducción

Un modelo probabilístico es una representación formal que describe un evento o fenómeno en términos de una distribución de probabilidad, en lugar de proporcionar una respuesta única como lo hacen los modelos deterministas. Estos modelos se utilizan para modelar procesos estocásticos, donde los resultados no están determinados de antemano y pueden variar con el tiempo o en función de las condiciones.

Cuando los principios que rigen un fenómeno son desconocidos o es demasiado complejo describirlos de manera computacionalmente eficiente, una forma de aproximar la solución es modelar su distribución de probabilidad a partir de muestras recopiladas del entorno. De esta manera, se pueden hacer predicciones y tomar decisiones basadas en una comprensión probabilística del fenómeno en cuestión.

3.4.2 Estimación de modelos probabilísticos

Suponiendo que existe una distribución de probabilidad real Q , se puede utilizar un modelo paramétrico P para aproximar a Q . Para evaluar la validez de nuestro modelo debemos tener un medio de evaluar las diferencias entre ambas distribuciones. Sabemos que ambas variables son estocásticas, esto es, no están definidas por un único valor comparable, sino que son distribuciones (funciones). Debemos disponer, por tanto, de una medida que nos permita comparar funciones. Una forma de hacer esto es realizar una abstracción del concepto de distancia para generalizarlo, no únicamente para medir distancias entre puntos, sino también para identificar distancias entre distribuciones de probabilidad. Para facilitar el proceso, puede ser necesario hacer algún tipo

de suposición sobre P , como que su muestreo sea manejable, de cara a asegurar que las muestras del modelo sean comparables con las muestras reales; que P tenga un gradiente manejable con respecto al muestreo de cara a poder utilizar técnicas de optimización y, finalmente, que tenga una función de probabilidad manejable de forma que pueda calcularse la probabilidad punto a punto.

3.4.3 GANs como modelos probabilísticos

Como ya sabemos, las GANs usan una combinación de dos redes neuronales para crear un modelo generativo con el objetivo de aprender a imitar una distribución real. El generador, como su nombre indica, genera una muestra modelo a partir de un vector aleatorio mediante el uso de una transformación determinista. El discriminador recibe muestras pertenecientes a la distribución real escogidas al azar y del modelo de generación y se entrena para diferenciarlas. Ambas redes están entrenadas de manera adversaria, una para generar imágenes lo más cercanas posible a la distribución real, y la otra para detectar las muestras procedentes del generador. Aunque las GANs no proporcionan un valor de probabilidad de la imagen generada, es de esperar que este valor, aunque no sea conocido, exista.

3.4.4 Distancia entre distribuciones de probabilidad

Existen diferentes enfoques para definir la distancia entre las distribuciones de probabilidad. Podemos diferenciar principalmente entre tres enfoques diferentes:

- 1) Métricas de probabilidad integral.
- 2) Reglas para puntuación.
- 3) f-divergencias.

Métricas de probabilidad integral

Abordado por primera vez en la publicación de Sriperumbudur y otros (2010), el punto clave de este método es que ambas distribuciones aparecen en forma de expectativa, lo que permite aproximarse a dicho valor mediante muestreo. La distancia de Wasserstein se deriva de estos métodos, y luego se usó con éxito como una distancia en Wasserstein-GAN (Arjovsky y otros, 2017).

Reglas para puntuación

En Gneiting y Raftery (2007) se propuso el enfoque de utilizar una puntuación para evaluar la adecuación de una distribución a otra. El punto es la optimización de la puntuación. La puntuación máxima se logra cuando ambas distribuciones son iguales.

f-divergencias

Abordado por primera vez en Ali y Silvey (1966), las distribuciones P y Q se requieren en forma de función de densidad. Las f-divergencias son una generalización de la divergencia Kullback-Leiber (Kullback y Leibler, 1951). La expectativa de la distribución Q se multiplica por una función convexa de la relación de verosimilitud entre P y Q . Por lo general, no se aplica directamente porque normalmente no se dispone de la distribución de los datos de forma explícita.

En Nguyen y otros (2010), Reid y otros (2011) y en Goodfellow y otros (2020) se desarrollaron métodos para usar f-divergencias en problemas donde P tiene forma de distribución y Q tiene forma de expectativa, y también donde ambas distribuciones tienen forma de expectativa. Esto permitió el uso de tales métricas en problemas donde solo se encuentran disponibles muestras de ambas distribuciones.

Desde un punto de vista matemático, podemos definir la f-divergencia, denotada como $D_f(P \parallel Q)$, como la distancia entre dos distribuciones de probabilidad P y Q que se puede calcular mediante la ecuación 14.

$$D_f(Q \parallel P) = \int_{\mathcal{X}} p(\mathbf{x}) f\left(\frac{q(\mathbf{x})}{p(\mathbf{x})}\right) d\mathbf{x} \quad (14)$$

donde f es una función convexa, denominada función generador (nada que ver con el generador de las GANs), que cumple la propiedad $f(1) = 0$, esto es, que en aquellos puntos donde las dos funciones son iguales la distancia es cero.

La distancia siempre es mayor que 0 y es únicamente igual a cero cuando las dos distribuciones coinciden en todo el dominio $\mathcal{X}$.

Estimación de las f-divergencias mediante muestreo

En Nguyen y otros (2010) se deriva un método variacional general para estimar las f-divergencias a partir del muestreo de las distribuciones P y Q .

Tabla 1. f-divergencias y generador asociado. Formas distintas de medir la distancia entre dos distribuciones de probabilidad P y Q

Nombre	$D_f(P \parallel Q)$	Generador $f(u)$
Total variation	$\frac{1}{2} \int p(\mathbf{x}) - q(\mathbf{x}) d\mathbf{x}$	$\frac{1}{2} u - 1 $
Kullback-Leibler	$\int p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x})} d\mathbf{x}$	$u \log u$
Reverse Kullback-Leibler	$\int q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})} d\mathbf{x}$	$-\log u$
Pearson χ^2	$\int \frac{(q(\mathbf{x}) - p(\mathbf{x}))^2}{p(\mathbf{x})} d\mathbf{x}$	$(u - 1)^2$
Neyman χ^2	$\int \frac{(p(\mathbf{x}) - q(\mathbf{x}))^2}{q(\mathbf{x})} d\mathbf{x}$	$\frac{(1-u)^2}{u}$
Squared Hellinger	$\int (\sqrt{p(\mathbf{x})} - \sqrt{q(\mathbf{x})})^2 d\mathbf{x}$	$(\sqrt{u} - 1)^2$
Jeffrey	$\int (p(\mathbf{x}) - q(\mathbf{x})) \log \left(\frac{p(\mathbf{x})}{q(\mathbf{x})} \right) d\mathbf{x}$	$(u - 1) \log u$
Jensen-Shannon	$\frac{1}{2} \int p(\mathbf{x}) \log \frac{2p(\mathbf{x})}{p(\mathbf{x}) + q(\mathbf{x})} + q(\mathbf{x}) \log \frac{2q(\mathbf{x})}{p(\mathbf{x}) + q(\mathbf{x})} d\mathbf{x}$	$-(u + 1) \log \frac{1+u}{2} + u \log u$
Jensen-Shannon weighted	$\int p(\mathbf{x}) \pi \log \frac{p(\mathbf{x})}{\pi p(\mathbf{x}) + (1-\pi)q(\mathbf{x})} + (1-\pi)q(\mathbf{x}) \log \frac{q(\mathbf{x})}{\pi p(\mathbf{x}) + (1-\pi)q(\mathbf{x})} d\mathbf{x}$	$\pi u \log u + (1-\pi + \pi u) \log(1-\pi + \pi u)$
GAN	$\int p(\mathbf{x}) \log \frac{2p(\mathbf{x})}{p(\mathbf{x}) + q(\mathbf{x})} + q(\mathbf{x}) \log \frac{2q(\mathbf{x})}{p(\mathbf{x}) + q(\mathbf{x})} d\mathbf{x} - \log 4$	$u \log u - (u + 1) \log(u + 1)$
α -divergence	$\frac{1}{\alpha(\alpha-1)} \int \left(p(\mathbf{x}) \left[\left(\frac{q(\mathbf{x})}{p(\mathbf{x})} \right)^\alpha - 1 \right] - \alpha(q(\mathbf{x}) - p(\mathbf{x})) \right) d\mathbf{x}$	$\frac{1}{\alpha(\alpha-1)} (u^\alpha - 1 - \alpha(u - 1))$

Del análisis de funciones convexas sabemos que toda función convexa f tiene un conjugado de Fenchel f^* que cumple que $f(u) = \sup_{t \in \text{dom}_{f^*}} \{tu - f^*(t)\}$, esto es, que cualquier función convexa puede ser representada como un conjunto de máximos puntuales de funciones lineales, siendo $f^*(t)$ el punto de intersección con el eje de y para cada punto t .

$$\begin{aligned}
 D_f(Q \parallel P) &= \int_{\mathcal{X}} p(\mathbf{x}) f\left(\frac{q(\mathbf{x})}{p(\mathbf{x})}\right) d\mathbf{x} \geq \\
 &\geq \sup_{T \in \mathcal{T}} \left(\int q(\mathbf{x}) T(\mathbf{x}) d\mathbf{x} - \int_{\mathcal{X}} p(\mathbf{x}) f^*(T(\mathbf{x})) d\mathbf{x} \right) = \\
 &= \sup_{T \in \mathcal{T}} \left(\mathbb{E}_{\mathbf{x} \sim Q}[T(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim P}[f^*(T(\mathbf{x}))] \right) \quad (15)
 \end{aligned}$$

Las expresiones pueden ser convertidas a expectativas y utilizadas en un algoritmo de muestreo. Un punto a tener en cuenta es que para algunas divergencias, f^* únicamente está definido para un dominio restringido. En esos casos, antes de utilizar la función objetivo, debe ser mapeada al dominio donde está definida.

Función variacional asociada al discriminador

En el artículo introductorio de las GANs (Goodfellow y otros, 2020) se demuestra que la optimización propuesta para la función de pérdida (eq. 7) es equivalente a minimizar la divergencia de Jensen-Shanon.

Comparando la expresión de la ecuación 15 con la función objetivo de GAN vemos que $T(x) = \log(D_\omega(x))$. Confirmamos pues que la GAN original minimiza la divergencia Jensen-Shanon, un caso particular de la f-GAN.

3.4.5 Conclusión

En este subapartado se ha generalizado el concepto de medida de la distancia entre distribuciones de probabilidad, utilizado en la función de pérdida de las GAN, para poderlo usar en un contexto más general con cualquier f-divergencia para la medida de la distancia entre la distribución de probabilidad del modelo y la original. En este contexto más general, se ha demostrado que la GAN original es un caso particular de optimización que utiliza la f-divergencia de Jensen-Shanon para la medida de la distancia entre distribuciones. Cualquiera de las medidas alternativas presentadas da lugar a optimizaciones equivalentes, a pesar de ser distintas, desde un punto de vista representativo.

3.4.6 Funciones de optimización mejoradas

En el anterior subapartado hemos introducido el concepto de f-divergencia para generalizar el concepto de distancia entre distribuciones y hemos nombrado otros métodos alternativos como las métricas de probabilidad integral. En este subapartado presentaremos las opciones de optimización que en la práctica han demostrado ser más efectivas para mejorar los problemas relacionados con la optimización min-max original, esto es, el colapso de modo y el desvanecimiento de gradientes. Los objetivos presentados, aparte de remediar estos aspectos, también mejoran la calidad de las imágenes generadas.

Wasserstein GAN (WGAN)

WGAN (Arjovsky y otros, 2017) resuelve el problema del desvanecimiento de gradientes y colapso de modo reemplazando la optimización de la f-divergencia de Jensen Shanon mediante el uso de la métrica de probabilidad integral EM [Earth mover distance (Hitchcock, 1941)], también llamada distancia de Wasserstein. La ecuación 16 muestra la ecuación que la define.

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [|x - y|] \quad (16)$$

donde $\Pi(p_r, p_g)$ representa el conjunto de todas las distribuciones conjuntas $\gamma(x,y)$ cuyos marginales son p_r y p_g . La distancia EM representa el mínimo coste necesario para transportar la «masa» de p_r a p_g para conseguir hacerlas iguales.

Las f-divergencias como KL y JS muestran problemas de inestabilidad cuando p_r y p_g están muy alejadas una de otra. EM es estable también en estos casos, además de ser continua, hecho que facilita la derivación de gradientes útiles para llevar a cabo la optimización. Un inconveniente es que el ínfimo de la ecuación 16 es intratable. Por esa razón los autores de WGAN estiman el coste de EM con la ecuación 17.

$$W(p_r, p_g) \approx \max_{w \in \mathcal{W}} \mathbb{E}_{\mathbf{x} \sim p_r} [f_w(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p_z} [f_w(G(\mathbf{z}))] \quad (17)$$

donde $f_{w \in \mathcal{W}}$ es una familia de funciones paramétricas que son K-Lipschitz para algún $K(\|f\|_L \leq K)$.

Los autores proponen encontrar la mejor función f_w que maximiza la ecuación 17 propagando el gradiente $\mathbb{E}_{\mathbf{z} \sim p_z} [f_w(G(\mathbf{z}))]$, donde g_θ es el generador g con parámetros θ . f_w puede ser representada por D pero condicionada a ser K-Lipschitz. w en f_w representa a los parámetros de D y el objetivo de D es maximizar 17, que aproxima la distancia EM. Cuando D es óptimo, 17 se aproxima a la distancia EM real y G se optimiza para minimizar la ecuación 18.

$$-\min_G \mathbb{E}_{\mathbf{z} \sim p_z} [f_w(G(\mathbf{z}))] \quad (18)$$

WGAN suele tener unos gradientes más suaves y medibles en todo el dominio que otras f-divergencias y aprende mejor incluso cuando no está produciendo aún buenas imágenes.

GAN autosupervisada (SSGAN)

SSGAN (Chen y otros, 2019) utiliza un sistema similar a la CGAN (Mirza y Osindero, 2014), pero sin la necesidad de contar con etiquetado explícito. Además, introduce un nuevo elemento en la función de pérdida, el objetivo del cual es prever el valor de una clase que se deriva directamente de la naturaleza de la imagen analizada. Concretamente, los autores predicen la rotación de la imagen de cuatro posibles valores. Este valor es conocido y calculable sin necesidad de etiquetar las imágenes y se demuestra útil para mejorar las capacidades predictivas del discriminador, así como para aprender representaciones útiles para la generación. La nueva función de pérdida tiene la forma indicada en la ecuación 19.

$$\begin{aligned} L_G &= -V(G, D) - \alpha \mathbb{E}_{\mathbf{x} \sim p_G} \mathbb{E}_{r \sim \mathbb{R}} [\log Q_D(R = r \mid \mathbf{x}^r)] \\ L_D &= -V(G, D) - \beta \mathbb{E}_{\mathbf{x} \sim p_{data}} \mathbb{E}_{r \sim \mathbb{R}} [\log Q_D(R = r \mid \mathbf{x}^r)] \end{aligned} \quad (19)$$

k-Lipschitz

Definición: Sea $A \subseteq \mathbb{R}$, $A \neq \emptyset$ y $k \in \mathbb{R}, k > 0$ decimos que una función $g : A \rightarrow \mathbb{R}$ es k-Lipschitz si $\forall x, y \in A |g(x) - g(y)| \leq k|x - y|$. Intuitivamente, es una condición más fuerte que la continuidad y limita el grado de variación permitido de g a un máximo fijado por la diferencia entre los dos valores x , y considerados multiplicados por la constante k .

donde $V(G,D)$ es el objetivo general de la GAN original (ecuación 6), P_{data} y P_G son las distribuciones real y del generador respectivamente, $r \in \mathbb{R}$ es la rotación seleccionada entre los ángulos permitidos ($R = \{0,90,180,270\}$). Una imagen rotada r grados se denota como $\mathbf{x}^r$ y $Q(R | \mathbf{x}^r)$ es la distribución predicha sobre los ángulos de rotación de la muestra. Esta función de pérdida fuerza a aprender representaciones internas de la rotación de forma supervisada, hipotetizando que dichas representaciones serán útiles también para la generalización de las capacidades generativas de la GAN. SSGAN obtiene resultados comparables a los de CGAN sin la necesidad de etiquetado externo.

Normalización espectral (SNGAN)

En SNGAN (Miyato y otros, 2018) se propone el uso de normalización de los parámetros como medio de estabilización del entrenamiento del discriminador. Es una técnica que es computacionalmente eficiente y que puede ser aplicada de manera sencilla. Sabemos que si D es una función k -Lipshitz, es decir, intuitivamente, que es una función continua sin variaciones abruptas, esto puede servir para estabilizar el entrenamiento de las GAN. SNGAN controla el valor de la constante de Lipschitz limitando la norma espectral de cada capa, normalizando la matriz de pesos de cada capa W para satisfacer la restricción $\sigma(W) = 1$ (esto es, que el mayor valor singular de la matriz de pesos sea 1). Esto se consigue simplemente normalizando cada capa mediante la ecuación 20.

$$\tilde{W}_{SN}(\mathbf{W}) = \frac{\mathbf{W}}{\sigma(\mathbf{W})} \quad (20)$$

donde $\mathbf{W}$ representa a la matriz de pesos de cada capa. En el artículo citado se demuestra que esta operación hace que la constante de Lipschitz en el discriminador quede limitada a un máximo de 1, hecho que facilita la optimización.

SNGAN consigue mejoras importantes de los resultados comparado con las técnicas previas de estabilización del entrenamiento publicadas, entre las que se incluyen el recorte del valor de los pesos (Arjovsky y otros, 2017), la penalización de gradientes (Wu y otros, 2019), (Mescheder y otros, 2018), normalización batch (Ioffe y Szegedy, 2015), normalización de pesos (Salimans y Kingma, 2016), normalización de capas (Lei Ba y otros, 2016) y regularización ortonormal (Brock y otros, 2016).

SphereGAN

SphereGAN (Park y Kwon, 2020) es una GAN que utiliza métrica de probabilidad integral (IPM) que usa una hiperesfera para ligar la IPM a la función

objetivo y de esta forma mejorar la estabilidad del entrenamiento. La función objetivo utilizada es la indicada en la ecuación 21.

$$\min_G \max_D \sum_r \mathbb{E}_x[d_s^r(\mathbf{N}, D(\mathbf{x}))] - \sum_r \mathbb{E}_z[d_s^r(\mathbf{N}, D(G(\mathbf{z})))] \quad (21)$$

para $r = 1, \dots, R$ donde d_s^r mide la distancia con momento r entre cada muestra y el polo norte de la hiperesfera, $\mathbf{N}$. El subíndice s indica que d_s^r está definido en $\mathbb{S}^n$.

Esta propuesta mejora las anteriores funciones objetivo basadas en la distancia Wasserstein gracias a la definición de los IPM en la hiperesfera. Esto le permite prescindir de muchas de las restricciones que deben ser impuestas a D para asegurar un entrenamiento estable en las anteriores propuestas, como las restricciones sobre la naturaleza Lischitz de las funciones requeridas por la distancia Wasserstein.

El funcionamiento de SphereGAN se podría describir de la forma siguiente: la red G genera datos a partir de un vector aleatorio z . Imágenes reales y otras procedentes de G se alimentan a la red discriminadora D que, a diferencia de las propuestas anteriores, da como resultado un vector de dimensión n . SphereGAN remapea este vector en una hiperesfera de n dimensiones utilizando transformaciones geométricas. Los puntos mapeados se utilizan para calcular los momentos geométricos centrados en el polo norte ($\mathbf{N}$) de dicha hiperesfera. La red discriminadora intenta maximizar las diferencias de momento entre las imágenes reales y las falsas, mientras que la generadora intenta conseguir lo contrario.

3.5 Una aplicación de las GAN: aumento de datos

El aumento de datos es una técnica habitual para mejorar el entrenamiento de los modelos en aquellas circunstancias donde se dispone de pocos datos. Podríamos decir que las circunstancias más habituales que requieren aumento de datos serían las siguientes:

- **Etiquetado limitado:** disponer de pocos datos etiquetados.
- **Diversidad limitada:** cuando el conjunto de entrenamiento carece de variedad en los datos.
- **Datos restringidos:** alguna de la información es sensible y no puede utilizarse directamente.

Los dos primeros casos se pueden resolver manualmente, poniendo recursos para etiquetar más datos, pero puede suponer un coste elevado. Una alterna-

tiva para el primer caso consiste en utilizar GANs para aumentar la cantidad de datos. SGAN, por ejemplo, permite generar nuevos datos anotados. El segundo caso es más habitual, pues suele ser habitual disponer de conjuntos de datos con clases desbalanceadas pobres en las clases raras.

3.6 Conclusiones

En este apartado hemos introducido a las GANs como método para la generación de datos. Es un tipo de arquitectura especialmente estudiada en el campo de generación de imagen. Es un tipo de arquitectura con mucho potencial que tiene algunos problemas sobre todo relacionados con la estabilidad en el entrenamiento. En este apartado hemos estudiado el diseño original, sus ventajas e inconvenientes, así como algunas modificaciones destinadas a solventar los problemas de la propuesta original. Si bien hemos intentado describir los puntos que consideramos clave, el campo en cuestión es muy amplio, tanto en arquitecturas como en aplicaciones. Este apartado debe servir como punto de partida para complementarlo con la bibliografía citada, así como con las futuras aportaciones.

4. Autocodificadores

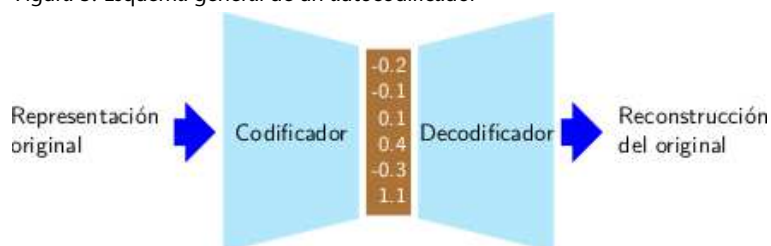
Antes de presentar los VAEs, consideramos apropiado brindar una breve introducción a la estructura general de los autocodificadores. Esta arquitectura precede en la historia a los variacionales y comparte muchos aspectos en común, aunque sus aplicaciones no se enfoquen en la generación de datos, como sucede con los VAEs.

4.1 Introducción a la estructura general de los autocodificadores

Los autocodificadores (Ackley y otros, 1985), (Hinton y Zemel, 1993), (Micheleluci, 2022) son un tipo de red neuronal especializada en la representación de un espacio dimensional de origen en otro más pequeño. El objetivo de esta transformación es la representación de la información observada en un vector de menor dimensión que lo represente con la mayor fidelidad posible. Se pretende que dicha compresión sirva para codificar en la red una representación significativa de los verdaderos factores explicativos de las señales observadas. A la representación vectorial reducida se le denomina espacio latente.

Los autocodificadores (figura 3) son una propuesta de arquitectura para llevar a cabo este proceso. Estos consisten en dos partes diferenciadas: un codificador que transforma el vector de datos inicial en una representación interna y un decodificador que a partir de esta última intenta reconstruir el original. Ambos elementos son redes neuronales cuya función es modelizar la distribución de probabilidad de los datos de origen. La arquitectura se optimiza de forma global. El cuello de botella existente en la zona del espacio latente es el que, idealmente, obliga a la arquitectura a comprimir en representaciones más sencillas la información relevante a partir de la cual se reconstruirá el original.

Figura 3. Esquema general de un autocodificador



4.2 Definición formal

Sea $\mathcal{X}$ el espacio de los datos de entrada y $\mathcal{Z}$ el de la representación codificada, ambos espacios euclidianos, esto es, $\mathcal{X} = \mathbb{R}^m$, $\mathcal{Z} = \mathbb{R}^n$, donde habitualmente $m > n$.

Definimos dos funciones paramétricas denominadas codificador $E_\phi : \mathcal{X} \rightarrow \mathcal{Z}$, parametrizada por ϕ y decodificador $D_\theta : \mathcal{Z} \rightarrow \mathcal{X}$, parametrizada por θ .

Para cualquier $x \in \mathcal{X}$, denotamos como $z = E_\phi(x)$ a la variable latente $z \in \mathcal{Z}$ derivada de aplicar la función E_ϕ a x y a $x' = D_\theta(z)$ como la reproducción de x mediante decodificación de z . Habitualmente las funciones paramétricas E_ϕ y D_θ son redes neuronales.

4.3 Entrenamiento

Para optimizar la arquitectura se debe definir un objeto matemático a optimizar. Para ello definimos inicialmente la función distancia d como $d : \mathcal{X} \times \mathcal{X} \rightarrow [0, \infty]$, tal que $d(x, x')$ es una función distancia que mide la diferencia entre ambos vectores.

Definida d , definiremos la función de optimización que minimiza la distancia sobre todo el conjunto de datos. Esto se puede hacer minimizando el valor esperado de la distancia sobre todo el conjunto de datos de origen:

$$L(\theta, \phi) = \arg \min_{\theta, \phi} \mathbb{E}_{x \sim \mu_{ref}} [d(x, D_\theta(E_\phi(x)))]$$

Para aquellos casos en que disponemos únicamente de un conjunto de datos finito representativo de $\mathcal{X}$, $x_1, \dots, x_N \in \mathcal{X}$, entonces $\mu_{ref} = \frac{1}{N} \sum_{i=1}^N \delta_{x_i}$.

Si se define d como $d(x, x') = \|x - x'\|^2$, entonces el problema se convierte en una optimización de mínimos cuadrados.

4.4 Autocodificadores regularizados

El objetivo último del autocodificador es conseguir una representación comprimida del vector de entrada, idealmente formada por sus factores constitutivos. Esto se pretende conseguir optimizando un objetivo matemático, aunque no siempre la optimización del segundo conduce al primero. Debido a que la optimización se realiza mediante el uso de una muestra finita de la población total, esto es, con un conjunto de entrenamiento finito, en determinadas circunstancias puede darse el caso que a la función paramétrica le sea más

sencilla la memorización del conjunto de datos que el aprendizaje de representaciones sencillas. En estos casos, decimos que el autocodificador aprende la función identidad. Este caso no es deseable, pues se aleja del objetivo que pretendemos conseguir.

Una forma de evitar esto es dotar a la función de optimización de mecanismos de regularización que eviten la memorización. Dependiendo de cómo se realice esta regularización, distinguimos entre distintas subclases de autocodificadores, entre los que se encuentran los autocodificadores poco densos, los de atenuación de ruido y los contractivos.

4.4.1 Autocodificadores poco densos (SAE)

Los autocodificadores poco densos introducen una restricción en la codificación, de manera que esta sea poco densa, en el sentido que únicamente se permita que unos pocos de los elementos de la representación latente estén activos. Hay dos formas básicas de conseguir este objetivo: la primera es fijar a cero todas las activaciones excepto las k mayores, donde k es un valor predefinido. A este tipo de autocodificador se le denomina autocodificador poco denso k (Makhzani y Frey, 2013); la segunda consiste en añadir a la función de pérdida original una función de regularización, esto es, $\min_{\theta, \phi} L(\theta, \phi) + \lambda L_{\text{sarsity}}(\theta, \phi)$, donde $\lambda > 0$ fija cuánta baja densidad queremos aplicar (Ng y otros, 2011). Para un autocodificador de K capas, una propuesta para la función de pérdida de regularización sería la siguiente:

$$L_{\text{sarsity}}(\theta, \phi) = \mathbb{E}_{x \sim \mu_x} \left[\sum_{k \in 1:K} \omega_k \|h_k\| \right]$$

donde h_k representa el valor del vector de la función de activación de la capa k y $\|\cdot\|$ normalmente representa la norma L1 o la norma L2, dando lugar respectivamente al autocodificador poco denso L1 o L2.

4.4.2 Autocodificadores de atenuación de ruido (DAE)

Los autocodificadores de atenuación de ruido (Vincent y otros, 2010) introducen la regularización modificando la función de reconstrucción. Durante el entrenamiento, la entrada se distorsiona y se intenta recuperar la información original.

El proceso de distorsión se hace mediante una función $T : \mathcal{X} \rightarrow \mathcal{X}$ que, a partir de una señal de entrada $x \in \mathcal{X}$, la transforma en una versión distorsionada $T(x)$. El valor de T se selecciona aleatoriamente con una distribución de probabilidad μ_T .

La función de optimización para el caso del DAE es:

$$\min_{\theta, \phi} L(\theta, \phi) = \mathbb{E}_{x \sim \mu_X, T \sim \mu_T} [d(x, (D_\theta \circ E_\phi \circ T)(x))]$$

Funciones típicas utilizadas para la adición de ruido incluyen el ruido isotrópico gaussiano, ruido mediante enmascaramiento aleatorio de parte de la entrada o fijar a valores aleatorios parte de los elementos de la entrada.

4.4.3 Autocodificador contractivo

Un autocodificador contractivo (Rifai y otros, 2011) añade una pérdida de regularización a la pérdida estándar del autocodificador:

$$\min_{\theta, \phi} L(\theta, \phi) + \lambda L_{contractive}(\theta, \phi)$$

donde λ regula la importancia de la regularización contractiva, que se define como la norma de Frobenius (Van Loan y Golub, 1996) esperada del jacobiano de las activaciones del codificador con respecto a la entrada:

$$L_{contractive}(\theta, \phi) = \mathbb{E}_{x \sim \mu_{ref}} \|\nabla_x E_\phi(x)\|_F^2$$

Sabemos que $\|E_\phi(x + \delta x) - E_\phi(x)\|_2 \leq \|\nabla_x E_\phi(x)\|_F \|\delta x\|_2$ para cualquier entrada $x \in \mathcal{X}$ y pequeña variación δx sobre ella. Por lo tanto, si $\|\nabla_x E_\phi(x)\|_F^2$ es pequeño, significa que valores alrededor de la entrada se asignan también a valores situados en un entorno próximo de su representación latente. Esta es una propiedad deseada, ya que significa que una pequeña variación de la entrada conduce a una pequeña variación, tal vez incluso cero, de su valor latente. Esto hace, por ejemplo, que dos imágenes puedan verse iguales incluso si no son exactamente iguales.

El DAE puede entenderse como un límite infinitesimal de CAE: en el límite del ruido de entrada gaussiano, los DAE hacen que la función de reconstrucción resista las perturbaciones de entrada, mientras que el CAE hace que las características extraídas resistan perturbaciones de entrada infinitesimales.

4.5 Aplicaciones típicas

Las aplicaciones típicas de los autocodificadores descritos en este subapartado incluyen las siguientes:

- Compresión: utilización de un autocodificador para representar la señal de entrada con otra de dimensión menor. A efectos comparativos, cuando un

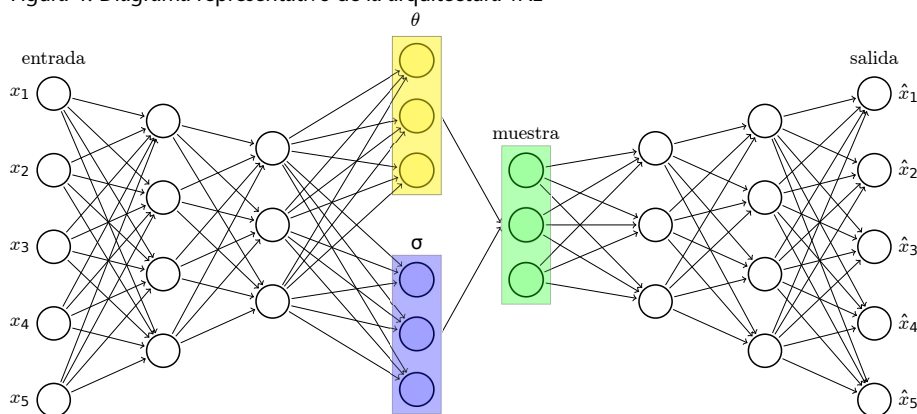
autocodificador usa una función lineal como codificador y decodificador, MSE como función de pérdida y los datos de entrada se normalizan de acuerdo a $\hat{x}_{i,j} = \frac{1}{\sqrt{M}} \left(x_{i,j} - \frac{1}{M} \sum_{k=1}^M x_{k,j} \right)$, entonces es equivalente a un PCA (Plaut, 2018).

- Clasificación mediante el uso de atributos latentes: los atributos del espacio latente pueden ser utilizados para clasificar imágenes, por ejemplo, utilizando k-NN para llevar a cabo la separación (supervisada o no supervisada) (Pulgar y otros, 2018).
- Detección de anomalías: se entrena un autocodificador en un conjunto de datos que contiene un conjunto representativo de las imágenes/datos que se quieren reconocer como válidas para luego calcular el error de reconstrucción de las imágenes/datos anómalos y compararlo con los errores de las imágenes/datos correctos. Si ambos conjuntos de imágenes/datos son suficientemente distintos, el error de reconstrucción de las instancias anómalas será superior al de las correctas. Fijando el umbral en el punto correcto será posible detectar aquellas que sean anómalas, pues su error de reconstrucción debería ser más alto (Chen y otros, 2017), (Zhou y Paffenroth, 2017), (Pang y otros, 2021).
- Atenuación de ruido en imágenes: el autocodificador se utiliza para corregir errores en la imagen de entrada (Vincent y otros, 2008), (Gondara, 2016).

5. Modelos generativos basados en autocodificadores variacionales (VAEs)

Los VAEs (figura 4) son una tipología de modelos gráficos probabilísticos basados en redes neuronales. Presentan una estructura similar a la de los autocodificadores introducidos en el apartado anterior. Permiten realizar inferencia eficiente en la presencia de variables latentes continuas (y/o discretas), con distribuciones intratables del posterior y utilizando conjuntos de datos de gran tamaño.

Figura 4. Diagrama representativo de la arquitectura VAE



Los VAEs son capaces de generar nuevos datos similares a los de la distribución de origen. Esto se consigue mediante la codificación, en forma de variables latentes, de las características definitorias de la distribución de probabilidad representada por la muestra poblacional del conjunto de datos original. A partir del modelo aprendido, son capaces de generar nuevas instancias que aproximan a dicha distribución.

Una red codificadora (también llamada red de reconocimiento) transforma la distribución de origen a un espacio latente estocástico representado por una distribución más sencilla (por ejemplo, una distribución normal). A partir de la distribución latente se generan los datos de reconstrucción mediante el uso de una red decodificadora (o red de generación).

Al final del entrenamiento, si este ha sido efectivo, será posible generar nuevas instancias mediante el decodificador, transformando las muestras del espacio oculto al espacio de expresión. El codificador actúa como un modelo de reconocimiento, mientras que el decodificador actúa como uno de generación. El modelo de reconocimiento proporciona al generador una aproximación del posterior sobre el espacio latente, que es utilizado para optimizar los paráme-

tros de ambas redes mediante estimación por máxima verosimilitud. Según la regla de Bayes, el modelo de reconocimiento aproxima a la inversa del modelo generativo.

5.1 Modelos probabilísticos

Los modelos probabilísticos son una representación matemática de fenómenos afectados de incertidumbre. Estos quedan definidos mediante distribuciones de probabilidad. Suponiendo que $p^*(\mathbf{x})$ represente la distribución de probabilidad original, podemos intentar aproximarla mediante un modelo paramétrico $p_\theta(\mathbf{x})$ tal que $p_\theta(\mathbf{x}) \approx p^*(\mathbf{x})$. Es de interés que $p_\theta(\mathbf{x})$ sea capaz de adaptarse a las características de los datos de origen, así como de incorporar información que conozcamos *a priori*, como por ejemplo, el conocimiento procedente de distribuciones de probabilidad condicionadas.

A veces puede suceder que no interese modelizar $p_\theta(\mathbf{x})$, sino que nos sea suficiente con modelizar $p_\theta(\mathbf{y}|\mathbf{x})$ con el objetivo de conseguir $p_\theta(\mathbf{y}|\mathbf{x}) \approx p^*(\mathbf{y}|\mathbf{x})$. Para simplificar la notación, en este apartado nos referiremos a la modelización incondicional, aunque lo explicado aquí también aplique a la condicional.

Los modelos gráficos probabilísticos (PGM) permiten representar la distribución conjunta de probabilidad atendiendo a las interconexiones causales existentes entre las distintas variables.

$$p_\theta(x_1, \dots, x_M) = \prod_{j=1}^M p_\theta(x_j | Pa(x_j))$$

donde $Pa(x_j)$ representa los nodos padres de x_j . Para los nodos sin padres, la probabilidad referida será incondicionada.

$p_\theta(x_j | Pa(x_j))$ puede ser definida de distintos modos, a saber, como una tabla de valores, como un modelo lineal o también como una red neuronal. En este último caso, $p_\theta(x | Pa(\mathbf{x})) = p_\theta(\mathbf{x}|\eta)$, donde $\eta = \text{RedNeuronal}(Pa(\mathbf{x}))$. Estos últimos son modelos paramétricos muy versátiles que permiten modelizar funciones complejas y son una muy buena herramienta para modelizar distribuciones optimizables matemáticamente.

5.2 Optimización de modelos probabilísticos

Sea un conjunto de datos $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \equiv \{\mathbf{x}^{(i)}\}_{i=1}^N = \mathbf{x}^{(1:N)}$, donde $\mathbf{x}^{(i)}$ son muestras, independientes e idénticamente distribuidas (i.i.d), pertenecientes a una distribución subyacente. El modelo paramétrico que aproxima la distribución conjunta de probabilidad será de la forma $p_\theta(\mathcal{D}) = \prod_{\mathbf{x} \in \mathcal{D}} p_\theta(\mathbf{x})$, que

en su versión logarítmica se expresa como $\log p_{\theta}(\mathcal{D}) = \sum_{\mathbf{x} \in \mathcal{D}} \log p_{\theta}(\mathbf{x})$. La forma típica de calcular θ es optimizar esta función, esto es, la estimación por máxima verosimilitud (MLE) de las instancias pertenecientes al conjunto de datos $\mathcal{D}$. El método MLE equivale a la minimización de la divergencia de Kullback-Leibler entre la distribución de los datos y la del modelo (Broniatowski, 2014). Disponiendo de la función de optimización MLE, ya es posible calcular los parámetros θ de la red neuronal utilizando los métodos de entrenamiento convencionales, por ejemplo, el descenso de gradiente estocástico.

5.3 Inferencia variacional (VI)

5.3.1 Variables observables vs. ocultas

Consideramos una *variable observable* cuando forma parte de aquellas de las que disponemos de su valor en los puntos que forman parte del conjunto de datos. Una *variable oculta o latente* es aquella de la que no tenemos información directa a través del conjunto de entrenamiento conocido. Esta formalización permite conectar la información percibida a través de variables observadas con las abstracciones propias de los modelos interpretativos, que serían las variables ocultas o latentes que queremos calcular. Así, por ejemplo, en un conjunto de datos formado por imágenes, las variables observables serían los píxeles de los que disponemos de información, mientras que las ocultas de interés podrían ser las abstracciones en forma de modelos del mundo que hacemos los seres inteligentes, por ejemplo, los distintos tipos de objetos presentes en la imagen.

5.3.2 Formalización matemática

Supongamos que partimos de una distribución de probabilidad representada por un conjunto de datos $\mathcal{D}$, con $\mathbf{x} \in \mathcal{D}$, de la que queremos inferir una serie de propiedades ocultas (por ejemplo, abstracciones), representadas por un conjunto de variables, originalmente desconocidas, que llamaremos $\mathbf{z}$. Sea $p(\mathbf{x}, \mathbf{z})$ la distribución conjunta de las variables observables y ocultas.

En estos casos, nos interesa realizar la inferencia acerca de las variables desconocidas, esto es, calcular $p(\mathbf{z}|\mathbf{x})$. Sabemos que este valor se puede representar como:

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$$

Calcular $p(\mathbf{z}|\mathbf{x})$ a partir de la igualdad bayesiana puede tener su dificultad, pues habitualmente alguna de las variables suele resultar intratable computacionalmente.

La inferencia variacional trata de resolver el problema mediante una aproximación, esto es, restringiendo el espacio de búsqueda a una familia de funciones sencillas, predefinida de origen, que llamaremos familia de funciones variacional $q_\phi(\mathbf{z}|\mathbf{x})$. Es común, aunque no requisito, que la familia escogida sea la de funciones gaussianas.

A partir de aquí, el problema original se convierte en uno de optimización, consistente en encontrar los parámetros ϕ óptimos restringidos a la familia de funciones $q_\phi(\mathbf{z}|\mathbf{x})$ que minimizan la distancia existente entre las distribuciones $p(\mathbf{z}|\mathbf{x})$ y $q_\phi(\mathbf{z}|\mathbf{x})$. Habitualmente, la función de medida de distancia entre distribuciones escogida es la divergencia de Kullback-Leibler:

$$D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})) = \mathbb{E}_{\mathbf{z} \sim q} \left[\log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] = \sum_{\mathbf{x} \in \mathcal{X}} q_\phi(\mathbf{z}|\mathbf{x}) \log \left(\frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right)$$

En este caso, el objetivo de la inferencia variacional es encontrar:

$$\hat{\theta} = \arg \min_{\phi} D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x}))$$

Existen muchos algoritmos distintos para abordar este problema, cada uno con sus ventajas e inconvenientes que dependen de la naturaleza de $p(\mathbf{x}, \mathbf{z})$ y de la familia subrogada de q .

La inferencia variacional define una estrategia genérica para abordar el problema, pero no un algoritmo concreto para conseguirlo.

5.3.3 Límite inferior de evidencia (ELBO)

Llegados a este punto, vamos a definir una variable nueva partiendo de $p(\mathbf{x})$ que nos será de utilidad para facilitar posteriormente la estrategia de optimización.

Para ello nos serviremos de una propiedad de las funciones convexas, denominada *desigualdad de Jensen*, que dice lo que sigue: sea f una función convexa en el dominio $\mathcal{X}$, entonces $\mathbb{E}[f(x)] \leq f(\mathbb{E}[x]), \forall x \in \mathcal{X}$. A efectos de nuestro caso concreto, si tomamos a f como la función logarítmica, tenemos que $\mathbb{E}[\log(x)] \leq \log(\mathbb{E}[x])$ para todo el dominio de la función.

Utilizando dicha propiedad, simplificamos la expresión de $\log p(\mathbf{x})$, creando un límite inferior:

$$\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \\
&= \log \int p(\mathbf{x}, \mathbf{z}) \frac{q_\phi(\mathbf{z}|\mathbf{x})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} = \\
&= \log \mathbb{E}_{\mathbf{z} \sim q} \frac{p(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} \geq \\
&\geq \mathbb{E}_{\mathbf{z} \sim q} \log \frac{p(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} = \\
&= \mathbb{E}_{\mathbf{z} \sim q} \log p(\mathbf{x}, \mathbf{z}) - \mathbb{E}_{\mathbf{z} \sim q} \log q_\phi(\mathbf{z}|\mathbf{x}) \\
&= ELBO = \mathcal{L}_{\theta, \phi}(\mathbf{x})
\end{aligned}$$

A dicho límite inferior lo llamamos *límite inferior de evidencia*, *ELBO* en inglés o $\mathcal{L}_{\theta, \phi}(\mathbf{x})$ en su expresión matemática en este texto.

El ELBO también se puede expresar de forma alternativa como:

$$\begin{aligned}
\mathcal{L}_{\theta, \phi}(\mathbf{x}) &= \mathbb{E}_{\mathbf{z} \sim q} \log p(\mathbf{x}, \mathbf{z}) - \underbrace{\mathbb{E}_{\mathbf{z} \sim q} \log q_\phi(\mathbf{z}|\mathbf{x})}_{\text{Entropía de } q} \\
&= \underbrace{\mathbb{E}_{\mathbf{z} \sim q} \log p(\mathbf{x}|\mathbf{z})}_{\text{- Pérdida de reconstrucción}} - \underbrace{D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x}))}_{\text{KL entre } q \text{ y el prior}}
\end{aligned}$$

5.3.4 ELBO como medio de optimizar q_ϕ

A continuación demostraremos que $D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})) = \log p(\mathbf{x}) - \mathcal{L}_{\theta, \phi}(\mathbf{x})$, esto es, que maximizando el ELBO estamos minimizando también la distancia entre las distribuciones $p(\mathbf{z}|\mathbf{x})$ y $q_\phi(\mathbf{z}|\mathbf{x})$ y, por lo tanto, consiguiendo el objetivo buscado de aproximar $p(\mathbf{z}|\mathbf{x})$ con la función más sencilla $q_\phi(\mathbf{z}|\mathbf{x})$.

Demostración:

$$\begin{aligned}
D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})) &= \mathbb{E}_{\mathbf{z} \sim q} \left[\log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] = \\
&= \mathbb{E}_{\mathbf{z} \sim q} \left[\log q_\phi(\mathbf{z}|\mathbf{x}) \right] - \mathbb{E}_{\mathbf{z} \sim q} \left[\log p(\mathbf{z}|\mathbf{x}) \right] = \\
&= \mathbb{E}_{\mathbf{z} \sim q} \left[\log q_\phi(\mathbf{z}|\mathbf{x}) \right] - \mathbb{E}_{\mathbf{z} \sim q} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})} \right] = \\
&= \mathbb{E}_{\mathbf{z} \sim q} \left[\log q_\phi(\mathbf{z}|\mathbf{x}) \right] - \mathbb{E}_{\mathbf{z} \sim q} \left[\log p(\mathbf{x}, \mathbf{z}) \right] + \mathbb{E}_{\mathbf{z} \sim q} \left[\log p(\mathbf{x}) \right] = \\
&= \log p(\mathbf{x}) - \left(\mathbb{E}_{\mathbf{z} \sim q} [\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q} [\log q_\phi(\mathbf{z}|\mathbf{x})] \right) = \\
&= \log p(\mathbf{x}) - \mathcal{L}_{\theta, \phi}(\mathbf{x})
\end{aligned}$$

Sabemos que el ELBO es un límite inferior de la evidencia, esto es:

$$\mathcal{L}_{\theta, \phi}(\mathbf{x}) = \log p_\theta(\mathbf{x}) - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})) \leq \log p_\theta(\mathbf{x})$$

A partir de esta ecuación podemos entender que maximizando el ELBO con respecto a los parámetros ϕ y θ se consiguen dos objetivos: por un lado, maximizar la probabilidad marginal $p_\theta(\mathbf{x})$ (nuestro modelo generativo se vuelve mejor) y, por otro, minimizar la divergencia KL de la aproximación $q_\phi(\mathbf{z}|\mathbf{x})$ sobre el posterior exacto $p_\theta(\mathbf{z}|\mathbf{x})$ (la aproximación $q_\phi(\mathbf{z}|\mathbf{x})$ mejora).

5.4 El codificador como forma de aproximar al posterior

Nuestro objetivo es aproximar al posterior, esto es:

$$q_\phi(\mathbf{z}|\mathbf{x}) \approx p_\theta(\mathbf{z}|\mathbf{x})$$

El modelo de inferencia puede ser un modelo del tipo:

$$q_\phi(\mathbf{z}|\mathbf{x}) = q_\phi(\mathbf{z}_1, \dots, \mathbf{z}_M|\mathbf{x}) = \prod_{j=1}^M q_\phi(\mathbf{z}_j|Pa(\mathbf{z}_j), \mathbf{x})$$

donde $Pa(\mathbf{z}_j)$ es el conjunto de nodos precedentes a la variable $\mathbf{z}_j$. La distribución $q_\phi(\mathbf{z}|\mathbf{x})$ puede ser igualmente modelada como una red neuronal profunda. De cara a encontrar los parámetros óptimos de la red neuronal, necesitamos definir una función objetivo tratable.

Utilizando la inferencia variacional definida en el subapartado anterior, restringiendo el espacio de búsqueda a la familia de funciones gaussianas, la función a optimizar tendría la forma siguiente:

$$(\mu, \log \sigma) = \text{RedNeuralCodificador}_\phi(\mathbf{x})$$

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu, \text{diag}(\sigma))$$

Esto es, la red neuronal serviría para calcular, para cada uno de los componentes de $\mathbf{z}$, la media μ y desviación estándar σ de una distribución normal que determinarían q_ϕ , siendo ϕ realmente los parámetros que definen a μ y σ .

En los métodos tradicionales de inferencia variacional, los parámetros se optimizan de manera separada para cada punto de datos. En el caso del autocodificador variacional, los parámetros se suponen comunes a todo el conjunto de datos. Esta aproximación al problema se denomina *inferencia variacional amortizada* (Gershman y Goodman, 2014) y permite la eliminación del bucle de optimización por punto de datos. Con esta suposición, implícitamente se está suponiendo que la expresividad de la red neuronal es suficiente para codificar la dependencia respecto a $\mathbf{x}$. Esto permite aprovechar la eficiencia del descenso de gradiente estocástico.

5.5 Optimización del ELBO mediante gradiente estocástico

Para poder utilizar al ELBO como función objetivo, es necesario derivar los gradientes respecto a θ y ϕ . Dado un conjunto i.i.d. de datos $\mathcal{D}$ el ELBO se puede representar como:

$$\mathcal{L}_{\theta,\phi}(\mathcal{D}) = \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}_{\theta,\phi}(\mathbf{x})$$

Normalmente, este objetivo suele ser intratable, aunque suele ser posible, tal como veremos a continuación, obtener buenos estimadores no sesgados que permitan llevar a cabo SGD.

El gradiente respecto a θ suele ser fácil de obtener:

$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{\theta,\phi}(\mathbf{x}) &= \nabla_{\theta} \mathbb{E}_q[\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\ &= \mathbb{E}_q[\nabla_{\theta}(\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x}))] \\ &= \mathbb{E}_q[\nabla_{\theta}(\log p_{\theta}(\mathbf{x}, \mathbf{z}))] \\ &\simeq \nabla_{\theta} \log p_{\theta}(\mathbf{x}, \mathbf{z}) \end{aligned}$$

La última línea corresponde con el estimador de la esperanza simple de Monte Carlo (Metropolis y Ulam, 1949).

No es tan sencillo para el caso de ∇_{ϕ} , pues tal como veremos en la derivación presentada a continuación, $\mathbb{E}_q$ depende de ϕ .

$$\begin{aligned} \nabla_{\phi} \mathcal{L}_{\theta,\phi}(\mathbf{x}) &= \nabla_{\phi} \mathbb{E}_q[\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\ &\neq \mathbb{E}_q \nabla_{\phi} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \end{aligned}$$

Para conseguir simplificar la esperanza, es necesario deshacer la dependencia de ϕ . Esto es posible hacerlo realizando un cambio de variable que desligue el muestreo de ϕ , esto es, convertir $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$ en $\epsilon \sim p(\epsilon)$ y expresar $\mathbf{z}$ como $\mathbf{z} = \mathbf{g}(\epsilon, \phi, \mathbf{x})$.

Realizando dicho cambio de variable, la esperanza puede ser expresada como sigue:

$$\begin{aligned} \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})}[f(\mathbf{z})] &= \mathbb{E}_{\epsilon \sim p(\epsilon)}[f(\mathbf{z})] \\ f(\mathbf{z}) &= \log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x}) \\ \mathbf{z} &= \mathbf{g}(\phi, \mathbf{x}, \epsilon) \end{aligned}$$

A partir de este cambio de variable, ya es posible derivar el estimador de Monte Carlo:

$$\begin{aligned}\nabla_{\phi} \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [f(\mathbf{z})] &= \nabla_{\phi} \mathbb{E}_{\epsilon \sim p(\epsilon)} [f(\mathbf{z})] \\ &= \mathbb{E}_{\epsilon \sim p(\epsilon)} \nabla_{\phi} [f(\mathbf{z})] \\ &\simeq \nabla_{\phi} [f(\mathbf{z})]\end{aligned}$$

Esta reparametrización $\mathbf{z} = \mathbf{g}(\phi, \mathbf{x}, \epsilon)$ también permite expresar al ELBO como sigue:

$$\begin{aligned}\mathcal{L}_{\theta, \phi}(\mathbf{x}) &= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\ &= \mathbb{E}_{\epsilon \sim p_{\epsilon}} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})]\end{aligned}$$

A partir de esta expresión podemos derivar un estimador simple de Monte Carlo $\tilde{\mathcal{L}}_{\theta, \phi}(\mathbf{x})$ del ELBO de un punto utilizando una única muestra de ruido ϵ de $p(\epsilon)$:

$$\begin{aligned}\epsilon &\sim p(\epsilon) \\ \mathbf{z} &= \mathbf{g}(\phi, \mathbf{x}, \epsilon) \\ \tilde{\mathcal{L}}_{\theta, \phi}(\mathbf{x}) &= \log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})\end{aligned}$$

Este estimador puede ser fácilmente calculado mediante librerías de cómputo con diferenciación automática, de forma que sea sencillo calcular las derivadas respecto a los parámetros θ y ϕ . El valor de $\nabla_{\phi} \tilde{\mathcal{L}}_{\theta, \phi}(\mathbf{x})$ se puede utilizar para optimizar el ELBO mediante descenso de gradiente (Kingma y Welling, 2013). El algoritmo 1 describe la operativa de optimización.

Algoritmo 1 Optimización estocástica del ELBO. A este procedimiento de optimización se le denomina AEVB, i.e. *autocodificación bayesiana variacional*

Entrada: $\mathcal{D}$, conjunto de datos de entrenamiento

Parámetros: $q_{\phi}(\mathbf{z}|\mathbf{x})$, modelo de inferencia

Parámetros: $p_{\theta}(\mathbf{x}, \mathbf{z})$, modelo generativo

- 1) $(\theta, \phi) \leftarrow$ Inicialización
 - 2) **mientras** SGD no converja **hacer**
 - 3) $\mathcal{M} \sim \mathcal{D}$ (seleccionar minibatch)
 - 4) $\epsilon \sim p(\epsilon)$ (ruido aleatorio distinto para cada muestra de $\mathcal{M}$)
 - 5) Calcular $\tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$ y sus gradientes $\nabla_{\theta, \phi} \tilde{\mathcal{L}}_{\theta, \phi}(\mathcal{M}, \epsilon)$
 - 6) Actualizar θ y ϕ utilizando el optimizador SGD
 - 7) **fin**
-

El gradiente utilizado es un estimador no sesgado del gradiente del ELBO. La media del muestreo sobre $\epsilon \sim p(\epsilon)$ equivale al gradiente del ELBO sobre el punto considerado.

5.6 Cálculo de q_ϕ

Para obtener el estimador del ELBO es necesario calcular la densidad $q_\phi(\mathbf{z}|\mathbf{x})$. El cálculo de este valor es sencillo si se escoge una transformación adecuada $\mathbf{g}$. La densidad $p(\epsilon)$ es conocida, pues corresponde con la densidad de la distribución escogida de antemano.

Si $\mathbf{g}(\cdot)$ es una función invertible, entonces las funciones de densidad de probabilidad de ϵ y $\mathbf{z}$ están relacionadas por la siguiente expresión¹:

$$\log q_\phi(\mathbf{z}|\mathbf{x}) = \log p(\epsilon) - \log d_\phi(\mathbf{x}, \epsilon)$$

donde:

$$\log d_\phi(\mathbf{x}, \epsilon) = \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \epsilon} \right) \right|$$

siendo

$$\frac{\partial \mathbf{z}}{\partial \epsilon} = \frac{\partial(z_1, \dots, z_k)}{\partial(\epsilon_1, \dots, \epsilon_k)} = \begin{pmatrix} \frac{\partial z_1}{\partial \epsilon_1} & \dots & \frac{\partial z_1}{\partial \epsilon_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_k}{\partial \epsilon_1} & \dots & \frac{\partial z_k}{\partial \epsilon_k} \end{pmatrix}$$

⁽¹⁾ Para la prueba consultar apartado 1.6.10 en Soch y otros (2020)

5.7 Posteriores gaussianos

Es común escoger un codificador gaussiano $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$:

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{RedNeuralCodificador}_\phi(\mathbf{x})$$

$$q_\phi(\mathbf{z}|\mathbf{x}) = \prod_i q_\phi(z_i|\mathbf{x}) = \prod_i \mathcal{N}(z_i; \mu_i, \sigma_i^2)$$

Que después de reparametrizar queda como:

$$\epsilon \sim \mathcal{N}(0, \mathbf{I})$$

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{RedNeuralCodificador}(\mathbf{x})$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \epsilon$$

donde $\odot$ representa el producto elemento a elemento. La transformación jacobiana de ϵ a $\mathbf{z}$ es:

$$\frac{\partial \mathbf{z}}{\partial \epsilon} = \text{diag}(\sigma)$$

El logaritmo del determinante del jacobiano, siguiendo las ecuaciones del apartado anterior para el caso gaussiano, se puede expresar como:

$$\log d_\phi(\mathbf{x}, \epsilon) = \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \epsilon} \right) \right| = \sum_i \log \sigma_i$$

y la densidad del posterior es:

$$\log q_\phi(\mathbf{z}|\mathbf{x}) = \sum_i \log \mathcal{N}(\epsilon_i; 0, 1) - \log \sigma_i$$

cuando $\mathbf{z} = g(\epsilon, \phi, \mathbf{x})$.

5.8 Posterior gaussiano con matriz de covarianza general

El posterior gaussiano factorizado puede ser extendido al caso gaussiano con covarianza completa:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

La reparametrización vendría dada por:

$$\epsilon \sim \mathcal{N}(0, \mathbf{I})$$

$$\mathbf{z} = \boldsymbol{\mu} + \mathbf{L}\epsilon$$

donde $\mathbf{L}$ es la matriz triangular superior (o inferior) con valores no nulos en la diagonal resultante de realizar la descomposición de Cholesky, esto es, $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^T$. Se propone este cambio de variable ya que de esta forma el cálculo del jacobiano es trivial: $\frac{\partial \mathbf{z}}{\partial \epsilon} = \mathbf{L}$, resulta ser el producto de los elementos de la diagonal:

$$\log d_\phi(\mathbf{x}, \epsilon) = \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \epsilon} \right) \right| = \sum_i \log L_{ii}$$

Y la densidad logarítmica del posterior se puede representar como:

$$\log q_{\phi}(\mathbf{z}|\mathbf{x}) = \log p(\epsilon) - \sum_i \log |L_{ii}|$$

Calculemos ahora la varianza de $\mathbf{z}$ para demostrar que se corresponde con la resultante de la descomposición de Cholesky:

$$\begin{aligned}\Sigma(\mathbf{z}) &= \mathbb{E}[(\mathbf{z} - \mathbb{E}[\mathbf{z}])(\mathbf{z} - \mathbb{E}[\mathbf{z}])^T] \\ &= \mathbb{E}[\mathbf{L}\epsilon(\mathbf{L}\epsilon)^T] = \mathbf{L}\mathbb{E}[\epsilon\epsilon^T]\mathbf{L}^T = \mathbf{L}\mathbf{L}^T\end{aligned}$$

Se hace notar que $\mathbb{E}[\epsilon\epsilon^T] = \mathbf{I}$ ya que $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.

Una manera de calcular los parámetros de la función q_{ϕ} sería utilizar una red neuronal para predecir μ, Σ . El problema de esta aproximación es que para calcular $\mathbf{L}$ a través de la descomposición de Cholesky, incurriríamos en un coste computacional $\mathcal{O}(n^3)$ para calcular $\mathbf{L}$. Una forma alternativa más económica computacionalmente es calcular directamente $\mathbf{L}$, por ejemplo, de la siguiente forma:

$$\begin{aligned}(\mu, \log \sigma, \mathbf{L}') &\leftarrow \text{RedNeuronalCodificador}_{\phi}(\mathbf{x}) \\ \mathbf{L} &\leftarrow \mathbf{L}_{mask} \cdot \mathbf{L}' + \text{diag}(\sigma)\end{aligned}$$

La red neuronal predice $\mu, \log \sigma$ y una matriz $\mathbf{L}'$. De esta última únicamente se consideran los elementos de debajo de la diagonal. Esto se hace aplicando una máscara $\mathbf{L}_{mask}$ para descartar los no necesarios. Este principio de derivación se utiliza en la variación de VAE denominada *flujo autorregresivo inverso* (Kingma y otros, 2016).

5.9 Desafíos de la arquitectura

Uno de los problemas típicos al utilizar la optimización estocástica de los autocodificadores variacionales es que durante el proceso de entrenamiento la red quede atrapada en una zona de equilibrio no óptimo que dificulte el progreso, dando como resultado redes deficientes. Distintas soluciones se han planteado para este problema, que quedan fuera del alcance de un apartado introductorio a los VAEs. Para el que tenga interés en profundizar sobre este aspecto, puede consultar, por ejemplo, Bowman y otros (2015), Larsen y otros (2016), Kingma y otros (2016).

Las características definitorias del método de los VAEs hacen que busquemos una aproximación $q_{\phi}(\mathbf{z}|\mathbf{x})$ a $p_{\theta}(\mathbf{z}|\mathbf{x})$. Cuando no es posible conseguir una bue-

na aproximación, ya sea porque la función $q_\phi(\mathbf{z}|\mathbf{x})$ no sea suficientemente expresiva para aproximar a $p_\theta(\mathbf{z}|\mathbf{x})$, o bien por problemas en la optimización, nos encontramos con la generación de imágenes borrosas típicas de este tipo de arquitecturas.

5.10 Posteriores no gaussianos

Una forma de mejorar el rendimiento de los autocodificadores variacionales es a través de la mejora de la flexibilidad del modelo de inferencia $q_\phi(\mathbf{z}|\mathbf{x})$. Mejorar la capacidad expresiva de la función tiene una incidencia directa en el ajuste del límite inferior de evidencia a la probabilidad marginal. Cualquier modificación del modelo de inferencia debe cumplir dos premisas: (1) que sea computacionalmente eficiente de calcular y diferenciar, y (2) computacionalmente eficiente de muestrear, pues ambas operaciones deben ser realizadas para cada punto del minibatch, para cada iteración de optimización. También es deseable que el algoritmo sea paralelizable para los elementos de $\mathbf{z}$, pues esto permite trabajar con dimensiones altas del vector de variables latentes.

5.11 Propuestas de extensión del autocodificador variacional original

En este subapartado enumeramos algunas de las arquitecturas derivadas del autocodificador variacional original (Kingma y Welling, 2013) que extienden sus características.

Entre ellas podemos encontrar las que siguen:

- Structured VAE (Salimans, 2016), Auxiliary Deep VAE (Maaløe y otros, 2016), Conditional VAE (Sohn y otros, 2015): estos tipos de VAE permiten condicionar el aprendizaje de los espacios latentes y la generación a distintos tipos de variables auxiliares.
- Wasserstein Autoencoders (WAE-MMD) (Tolstikhin y otros, 2017), Sliced-Wasserstein VAE (SWAE) (Kolouri y otros, 2018): VAEs que miden la distancia entre distribuciones mediante la distancia Wasserstein en vez de la divergencia Kullback-Leibler.
- β -VAE (Higgins y otros, 2016), (Burgess y otros, 2018), (Chen y otros, 2018b): introduce una constante β que se utiliza durante el proceso de entrenamiento para modificar el peso de la divergencia de Kullback-Leibler en la función objetivo. Esto incide en la capacidad de separación de las características representadas por las variables latentes.
- Importance Weighted Autoencoders (IWAE) (Burda y otros, 2015), MIWAE (Rainforth y otros, 2018): utilizan una estimación más precisa del ELBO ba-

sada en el cálculo de varios valores de ϵ por muestra. Esto permite obtener un ELBO más ajustado a $p(\mathbf{z}|\mathbf{x})$.

- Deep Feature Consistent VAE (DFCVAE) (Hou y otros, 2017): añade a la función de pérdida de una VAE convencional la suma de los MSEs obtenidos de comparar las activaciones de la imagen original y la imagen reconstruida al paso de una CNN preentrenada. El objetivo es reducir la apariencia borrosa de las imágenes predichas por el VAE.
- MS-SSIM-VAE (Snell y otros, 2017) utiliza funciones de pérdida basadas en el uso de métricas de similitud perceptiva (combinación de luminancia, contraste y estructura).
- Categorical VAE (Jang y otros, 2016): propone una implementación con el uso de variables latentes categóricas mediante una aproximación diferenciable a su distribución de probabilidad.
- Joint-VAE (Dupont, 2018): propone el uso combinado de variables latentes continuas y discretas.
- Info-VAE (Zhao y otros, 2017): propone una función de optimización alternativa para abordar el problema típico de los VAEs cuando el generador obvia parte de la información aprendida para el codificador, generando imágenes subóptimas. Para esto introduce un término cuyo objetivo es maximizar la información mutua entre el espacio latente y el espacio de generación.
- LogCosh-VAE (Chen y otros, 2018a): propone el uso de una función de pérdida alternativa basada en el logaritmo del coseno hiperbólico para mejorar la calidad visual de las imágenes reconstruidas.
- DIP-VAE (Kumar y otros, 2017) Propuesta alternativa para la separación de las variables latentes del VAE para datos no etiquetados.
- Vector Quantized Variational Autoencoder (VQ-VAE) (Van Den Oord y otros, 2017): utiliza también un espacio latente discreto. Utiliza un «libro de códigos» para almacenar los valores permitidos para el vector del espacio latente. Los vectores presentes en el libro de códigos son aprendidos mediante descenso de gradiente.
- Normalizing Flows (Rezende y Mohamed, 2015), (Kobyzev y otros, 2020) se construye una distribución flexible del posterior a través de un procedimiento iterativo. Se comienza con una variable aleatoria con una distribución relativamente simple con una conocida (y computacionalmente económica) función de densidad de probabilidad, para luego derivar por iteración una cadena de transformaciones invertibles parametrizadas, de forma que la última iteración tenga una distribución más flexible. Esta operativa no escala bien para espacios dimensionales grandes.

- Inverse Autoregressive Flow (IAF) (Kingma y otros, 2016). Al igual que los flujos de normalización, se empieza con una distribución de probabilidad tratable (por ejemplo, con una gaussiana con covarianza diagonal), seguido de una serie de transformaciones invertibles no lineales de $\mathbf{z}$. Se espera que la iteración final tenga una distribución flexible.

En Subramanian (2020) podéis encontrar implementaciones de algunos de estos autocodificadores variacionales.

5.12 Conclusiones

Los modelos probabilísticos son un pilar fundamental en el campo de la inteligencia artificial moderna. Los modelos probabilísticos con variables latentes permiten asociar variables observables con otras ocultas. Estos pueden utilizarse para descubrir abstracciones, en forma de variables latentes, que sirvan para interpretar las regularidades estadísticas observadas en los datos. Las distribuciones de probabilidad derivadas de relacionar dichas variables con las observables pueden ser usadas a modo de función objetivo para el diseño de modelos predictivos de los datos procedentes de dichas distribuciones. El problema habitual en estos casos es que las relaciones entre distribuciones suelen implicar cálculos intratables desde un punto de vista computacional. La inferencia variacional (VI) permite convertir estos problemas intratables en otros tratables mediante optimización. La distribución de probabilidad que relaciona variables ocultas con observables, esto es, el posterior $p(\text{oculta}|\text{datos})$, se aproxima con una distribución de probabilidad más sencilla (por ejemplo, gaussiana), y se optimiza para que la distancia entre esta y el posterior real sea lo más pequeña posible. Esto se hace introduciendo unos nuevos modelos paramétricos denominados variacionales. Las redes neuronales, gracias a su alta flexibilidad y expresividad, son un candidato perfecto para su optimización en estas circunstancias.

La inferencia variacional en los VAEs realiza la optimización típicamente a través de un límite inferior de la evidencia (ELBO) mediante optimización por descenso de gradiente estocástico. El VAE resulta ser una combinación de un modelo profundo de variables latentes continuas y un modelo de inferencia asociado. El modelo de inferencia, denominado codificador o modelo de reconocimiento, aproxima la distribución del posterior. Tanto el modelo generativo como el de inferencia son modelos probabilísticos basados en redes neuronales. Los parámetros de ambos modelos son optimizados de manera conjunta realizando descenso de gradiente sobre el límite inferior de evidencia (ELBO). El ELBO es un límite inferior sobre la probabilidad marginal de los datos, también llamado límite inferior variacional. Los gradientes estocásticos necesarios para la optimización son obtenidos mediante una reparametrización.

Modificaciones posteriores se han realizado sobre el modelo inicial tanto para aumentar la expresividad del modelo generativo como para hacer más flexible el modelo de inferencia del posterior. Las primeras están destinadas a aumentar la variabilidad de los datos generados para que incluyan toda la diversidad ya codificada en las variables latentes aprendidas por el modelo; las segundas están destinadas a aumentar la flexibilidad del modelo de inferencia para hacer que el límite inferior de evidencia más próximo a la probabilidad marginal de los datos y, por tanto, aumentar la representatividad de variables latentes. Otro tipo de propuestas están destinadas a limitar la correlación entre los elementos del vector de variables latentes o incluso a usar una combinación de variables latentes discretas y continuas.

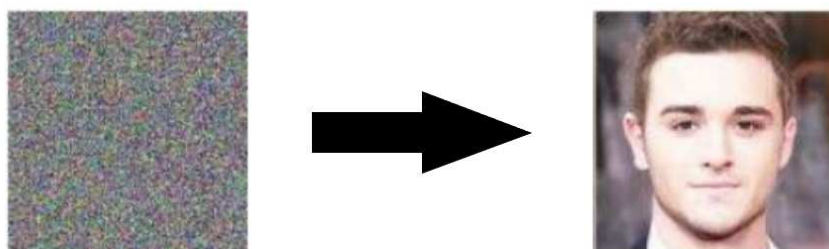
Los VAEs se mantienen como una arquitectura de referencia para la derivación de modelos de inferencia de variables latentes asociadas a conjuntos de variables observadas continuas y discretas, y para la síntesis de nuevas instancias pertenecientes a distribuciones de probabilidad próximas a la original.

6. Modelos generativos basados en difusión

Los modelos generativos basados en difusión, propuestos por primera vez en Sohl-Dickstein y otros (2015), son un marco de diseño que permite generar nuevas imágenes mediante la reversión de procesos análogos a los encontrados en la termodinámica de no equilibrio. Si en un vaso de agua introducimos un líquido miscible de un color distinto, observaremos un proceso físico de difusión en el que los dos líquidos se van mezclando progresivamente hasta finalmente formar una mezcla homogénea. Los modelos basados en difusión modelizan la reversión de este proceso.

Los modelos de difusión se pueden aplicar a señales de distinta naturaleza, entre otras, a señales de audio e imagen. Para el caso de imagen, el proceso se inicia llevando a cabo un proceso de corrupción progresiva de los píxeles que la componen mediante la aplicación de ruido aleatorio, a modo de proceso de difusión natural. Una red neuronal es entrenada para la reversión de cada uno de los mencionados pasos de corrupción. Para que el proceso de reconstrucción resulte reversible es necesario realizar la corrupción de forma muy progresiva. Es habitual que el número de pasos necesarios sea del orden del millar. Si el entrenamiento de dicha red es exitoso será posible generar una imagen a partir de ruido aleatorio encadenando un número de pasos similar al utilizado para la deconstrucción de imágenes en tiempo de entrenamiento.

Figura 5. Representación de la generación de imagen típica de un modelo generativo basado en mecanismos de difusión.



6.1 Formulación matemática

Los modelos de difusión son modelos generativos, esto es, son utilizados para generar nuevos datos. El mecanismo de funcionamiento consiste en dos etapas: una primera donde se procede a la corrupción incremental de los datos de entrada mediante la adición de ruido gaussiano, y una segunda donde

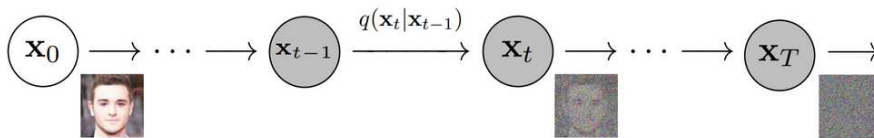
se aprende a revertir cada uno de los estadios de corrupción aplicados previamente. Después del entrenamiento, las redes de reconstrucción entrenadas son capaces de generar nuevos datos a partir de ruido gaussiano mediante la aplicación sucesiva del proceso de reconstrucción previamente entrenado.

Una cadena de Markov es un proceso estocástico en el cual la probabilidad de que suceda un evento depende únicamente del estado precedente, es decir, toda la información necesaria para predecir el estado siguiente está contenida en el estado actual, siendo, por tanto, independiente de todos los anteriores.

Un modelo de difusión puede ser modelizado como una cadena de Markov donde los nodos representan los sucesivos estadios de la reconstrucción de los datos desde su estadio original hasta el final, representativo de los datos reconstruidos.

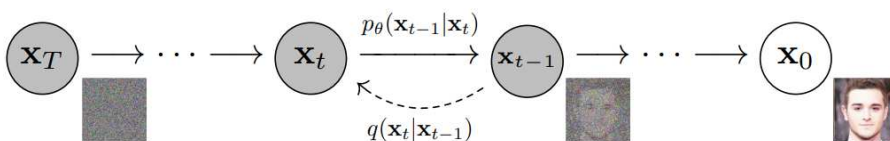
Para revertir el proceso debemos ser capaces de modelizar $q(\mathbf{x}_t|\mathbf{x}_{t-1})$. La figura 6 representa el proceso al que se somete cada imagen durante el entrenamiento. La cadena de Markov del proceso de difusión directa realiza una deconstrucción paulatina de los datos, añadiendo ruido de forma gradual, para obtener una aproximación al posterior $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$, donde $\mathbf{x}_1, \dots, \mathbf{x}_T$, son variables latentes de las mismas dimensiones que $\mathbf{x}_0$. Al final del proceso se acaba transformando la imagen original en ruido gaussiano.

Figura 6. Representación de la cadena de Markov representativa del proceso de deconstrucción de una imagen



La red generativa utiliza la información generada como medio de optimización del modelo de reconstrucción del proceso inverso. La función generativa corresponde a la distribución de probabilidad $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. La cadena de Markov representativa del proceso de reconstrucción se muestra en la figura 7.

Figura 7. Representación de la cadena de Markov representativa del proceso de reconstrucción de una imagen



Los modelos basados en difusión son capaces de generar imágenes de alta calidad sin requerir entrenamiento adversario típico de las GANs. Las dificultades de la optimización de las redes GAN ya ha sido tratada en el apartado corres-

pondiente. Los modelos de difusión presentan beneficios de escalabilidad y paralelización respecto a otros tipos de modelos generativos como el mencionado.

6.2 Modelos probabilísticos de eliminación del ruido producido por difusión

Los modelos de difusión son modelos de variables latentes de la forma $p_\theta(\mathbf{x}_0) = \int p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T) d\mathbf{x}_1 \dots d\mathbf{x}_T$, donde $\mathbf{x}_1 \dots \mathbf{x}_T$ son variables latentes de las mismas dimensiones que los datos $\mathbf{x}_0 \sim q(\mathbf{x}_0)$.

6.2.1 Proceso directo de difusión

Dado un punto de muestreo $\mathbf{x}_0 \sim q(\mathbf{x})$, definimos el proceso directo de difusión como aquel en el que añadimos ruido gaussiano a la muestra original en T pasos sucesivos, produciendo una secuencia de muestras $\mathbf{x}_1, \dots, \mathbf{x}_T$. El tamaño del cambio se controla mediante el parámetro $\{\beta_t \in (0, 1)\}_{t=1}^T$.

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$$

La muestra $\mathbf{x}_0$ pierde gradualmente sus atributos distinguibles. Con $T \rightarrow \infty$, $\mathbf{x}_T$ tiende a una distribución isotrópica gaussiana, por ejemplo, $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Una propiedad interesante que a continuación demostramos es que para cualquier t es posible obtener en un único paso la muestra $\mathbf{x}_t$:

Sea $\alpha_t = 1 - \beta_t$ y $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ entonces,

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}; & \boldsymbol{\epsilon}_i &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\boldsymbol{\epsilon}}_{t-2}; & \bar{\boldsymbol{\epsilon}}_{t-2} &\sim \mathcal{N}(\mathbf{0}, \sqrt{1 - \alpha_t \alpha_{t-1}}) \\ &= \dots \\ &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon} \\ q(\mathbf{x}_t | \mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \end{aligned}$$

Por lo general, conforme se avanza en el proceso de corrupción, es posible realizar pasos de actualización más grandes, por lo que $\beta_1 < \beta_2 < \dots < \beta_T$ y, por tanto, $\bar{\alpha}_1 > \bar{\alpha}_2 > \dots > \bar{\alpha}_T$.

Hasta ahora hemos visto cómo derivar $q(\mathbf{x}_t, \mathbf{x}_0)$. Para derivar $q(\mathbf{x}_t)$ sabemos que $q(\mathbf{x}_t) = \int q(\mathbf{x}_0, \mathbf{x}_t) d\mathbf{x}_0 = \int q(\mathbf{x}_t | \mathbf{x}_0) q(\mathbf{x}_0) d\mathbf{x}_0$, entonces podemos muestrear $\mathbf{x}_t \sim q(\mathbf{x}_t)$ en dos pasos: primero muestreando $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ de los datos de entrada y después muestreando $\mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)$, esto es, muestreo ancestral.

Suma de dos gaussianas

La suma de dos distribuciones gaussianas $\mathcal{N}(0, \sigma_1^2 I)$ y $\mathcal{N}(0, \sigma_2^2 I)$ es $\mathcal{N}(0, (\sigma_1^2 + \sigma_2^2) I)$

A continuación vamos a derivar la expresión de $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$. Combinando la aplicación de la regla de Bayes y la expresión de la función gaussiana tenemos:

$$\begin{aligned} q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) &= q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)} \\ &\propto \exp \left(-\frac{1}{2} \left(\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\alpha_{t-1}} \mathbf{x}_0)^2}{1 - \alpha_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_0)^2}{1 - \alpha_t} \right) \right) \\ &= \exp \left(-\frac{1}{2} \left(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t} \mathbf{x}_t \mathbf{x}_{t-1} + \alpha_t \mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\alpha_{t-1}} \mathbf{x}_0 \mathbf{x}_{t-1} + \alpha_{t-1} \mathbf{x}_0^2}{1 - \alpha_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_0)^2}{1 - \alpha_t} \right) \right) \\ &= \exp \left(-\frac{1}{2} \left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \alpha_{t-1}} \right) \mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{2\sqrt{\alpha_{t-1}}}{1 - \alpha_{t-1}} \mathbf{x}_0 \right) \mathbf{x}_{t-1} + C(\mathbf{x}_t, \mathbf{x}_0) \right) \right) \end{aligned}$$

Expresando los términos en función de una distribución estándar gaussiana tenemos que:

$$\begin{aligned} \tilde{\beta}_t &= 1 / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \alpha_{t-1}} \right) = 1 / \left(\frac{\alpha_t - \tilde{\alpha}_t + \tilde{\beta}_t}{\beta_t(1 - \tilde{\alpha}_{t-1})} \right) = \frac{1 - \tilde{\alpha}_{t-1}}{1 - \tilde{\alpha}_t} \cdot \beta_t \\ \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) &= \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\alpha_{t-1}}}{1 - \alpha_{t-1}} \mathbf{x}_0 \right) / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \alpha_{t-1}} \right) \\ &= \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\alpha_{t-1}}}{1 - \alpha_{t-1}} \mathbf{x}_0 \right) \frac{1 - \tilde{\alpha}_{t-1}}{1 - \tilde{\alpha}_t} \cdot \beta_t \\ &= \frac{\sqrt{\alpha_t(1 - \tilde{\alpha}_{t-1})}}{1 - \tilde{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1 - \tilde{\alpha}_t} \mathbf{x}_0 \end{aligned}$$

Sabemos que $\mathbf{x}_0 = \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t - \sqrt{1 - \alpha_t} \epsilon_t)$, introduciendo esta expresión en la de $\tilde{\mu}_t$ obtenemos:

$$\begin{aligned} \tilde{\mu}_t(\mathbf{x}_t, \epsilon_t) &= \frac{\sqrt{\alpha_t(1 - \tilde{\alpha}_{t-1})}}{1 - \tilde{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\alpha_{t-1}} \beta_t}{1 - \tilde{\alpha}_t} \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t - \sqrt{1 - \alpha_t} \epsilon_t) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_t \right) \end{aligned}$$

A pesar de que $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ es intratable, se demuestra que $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ es tratable, $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$.

6.2.2 Proceso de difusión inverso o de reconstrucción

Si fuera posible revertir el proceso descrito en el subapartado anterior y obtener muestras de la distribución $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ entonces sería posible reconstruir una muestra real a partir de una entrada de ruido gaussiano, $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Una forma de hacerlo es estimar $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ mediante el uso de un modelo paramétrico p_θ que aproxime dichas probabilidades condicionales. Para β_t suficientemente pequeño, $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ se puede aproximar a una distribución nor-

mal. Se decide aproximar $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ mediante un modelo paramétrico p_θ como una distribución gaussiana y parametrizar su media y su varianza:

$$\begin{aligned} p(\mathbf{x}_T) &= \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \\ p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) &= \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \\ \Sigma_\theta(\mathbf{x}_t, t) &= \sigma_t \mathbf{I} \end{aligned}$$

$\mu_\theta(\mathbf{x}_t, t)$ se suele parametrizar como una red neuronal (comúnmente utilizando la arquitectura U-Net o bien el autocodificador atenuador de ruido).

Si aplicamos la ecuación de inversión para todos los pasos (trayectoria) podemos ir de $\mathbf{x}_T$ hasta la distribución de los datos:

$$p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T) = p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

6.2.3 Función de pérdida

Para poder encontrar los parámetros óptimos del modelo de inversión, debemos buscar una forma de conectar el proceso de difusión directa (representado por la distribución q) con el proceso inverso (distribución p_θ). Una función objetivo podría ser la siguiente:

$$-\log p(\mathbf{x}_0) \leq -\log p(\mathbf{x}_0) + D_{KL}(q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0) || p_\theta(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0))$$

En el caso ideal que $q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)$ y $p_\theta(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)$ fueran iguales, la desigualdad se convertiría en igualdad.

Manipulando la expresión anterior:

$$\begin{aligned} -\log p(\mathbf{x}_0) &\leq -\log p(\mathbf{x}_0) + D_{KL}(q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0) || p_\theta(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)) \\ &= -\log p(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_1, \dots, \mathbf{x}_T \sim q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)/p_\theta(\mathbf{x}_0)} \right] \\ &= -\log p(\mathbf{x}_0) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)} + \log p_\theta(\mathbf{x}_0) \right] \\ &= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_1, \dots, \mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)} \right] = L_{VLB} \\ L_{VLB} &\geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0) \end{aligned}$$

Se puede llegar al mismo resultado partiendo de la desigualdad de Jensen:

$$\begin{aligned}
 L_{CE} &= -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0) \\
 &= -\mathbb{E}_{q(\mathbf{x}_0)} \log \left(\int p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T) d\mathbf{x}_1 \dots d\mathbf{x}_T \right) \\
 &= -\mathbb{E}_{q(\mathbf{x}_0)} \log \left(\int q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0) \frac{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)}{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} d\mathbf{x}_1 \dots d\mathbf{x}_T \right) \\
 &= \mathbb{E}_{q(\mathbf{x}_0)} \log \left(\mathbb{E}_{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \frac{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)}{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \right) \\
 &\leq -\mathbb{E}_{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \log \left(\frac{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)}{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)} \right) \\
 &= \mathbb{E}_{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \log \left(\frac{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)} \right) = L_{VLB}
 \end{aligned}$$

Para convertir cada término de la ecuación para que sea analíticamente computable, el objetivo se puede reescribir como una combinación de varios términos de divergencia KL y entropía [apéndice B de Sohl-Dickstein y otros (2015)] llegando a la siguiente expresión:

$$\begin{aligned}
 L_{VLB} &= \mathbb{E}_{q(\mathbf{x}_0, \dots, \mathbf{x}_T)} \left[\log \frac{q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0, \dots, \mathbf{x}_T)} \right] \\
 &= \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right] \\
 &= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right] \\
 &= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\
 &= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \left(\frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} \right) + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\
 &= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\
 &= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T | \mathbf{x}_0)}{q(\mathbf{x}_1 | \mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1 | \mathbf{x}_0)}{p_\theta(\mathbf{x}_0 | \mathbf{x}_1)} \right] \\
 &= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T | \mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0 | \mathbf{x}_1) \right] \\
 &= \mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right]
 \end{aligned}$$

donde $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ es la versión tratable de la distribución del posterior.

Etiquetando cada componente del ELBO separadamente tenemos:

$$L_{VLB} = L_T + L_{T-1} + \dots + L_0$$

donde

$$L_T = D_{KL}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))$$

$$L_t = D_{KL}(q(\mathbf{x}_t | \mathbf{x}_{t+1}, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_t | \mathbf{x}_{t+1})) \text{ para } 1 \leq t \leq T-1$$

$$L_0 = -\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)$$

Analicemos cada uno de los términos:

- 1) $\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$ Puede ser interpretado como un término de reconstrucción similar al ELBO de un autocodificador variacional. En Ho y otros (2020) modelizan L_0 utilizando un decodificador discreto separado derivado de $\mathcal{N}(\mathbf{x}_0; \mu_\theta(\mathbf{x}_1, 1), \Sigma_\theta(\mathbf{x}_1, 1))$
- 2) $D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))$ muestra cuán cerca se encuentra $\mathbf{x}_T$ de la función gaussiana estándar. El término no tiene parámetros entrenables y, por tanto, es ignorado durante el entrenamiento.
- 3) $D_{KL}(q(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1}))$ para $1 \leq t \leq T-1$, calcula la diferencia entre cada uno de los pasos inversos $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ y los aproximados $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$

Todos los términos de L_{VLB} (excepto L_0) establecen comparaciones entre distribuciones gaussianas y, por tanto, pueden ser calculadas de forma exacta. A través del ELBO, maximizar la probabilidad se reduce a aprender los pasos de eliminación del ruido L_t .

Como tanto $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ como $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ son distribuciones normales, la divergencia de Kullback-Leibler se puede expresar de forma analítica como:

$$L_{t-1} = D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C$$

Sabemos que $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1-\alpha_t}\epsilon$ y que $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon \right)$, entonces se puede representar la función paramétrica $\mu_\theta(\mathbf{x}_t, t)$ como un modelo paramétrico de predicción del ruido $\epsilon_\theta(\mathbf{x}_t, t)$:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

La función de pérdida quedaría representada de la forma siguiente:

$$\begin{aligned} L_{t-1} &= \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\lambda_t \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] + C \\ \lambda_t &= \frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\alpha_t)} \\ \mathbf{x}_t &= \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1-\alpha_t}\epsilon \end{aligned}$$

El valor de λ_t es un peso que pondera la importancia en función del tiempo. Este valor suele ser bastante grande para valores de t pequeños. En Ho y otros (2020) observan que estableciendo $\lambda_t = 1$ mejora la calidad del muestreo. Alineado con esta decisión, usan la función de pérdida siguiente:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), l \sim \mathcal{U}(1, T)} \left[\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right]$$

Otras estrategias avanzadas de ponderación de λ_t pueden consultarse en Choi y otros (2022).

6.2.4 Proceso de muestreo y entrenamiento

Los algoritmos 2 y 3 presentados a continuación describen el proceso de entrenamiento y de muestreo, respectivamente, presentado en este subapartado.

Algoritmo 2 Difusión: Algoritmo de entrenamiento

- 1) **repetir**
 - 2) $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3) $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4) $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5) Descenso de gradiente con $\nabla_{\theta} \|\epsilon_t - \epsilon_{\theta}(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t)\|^2$
 - 6) **hasta** convergencia
-

Algoritmo 3 Difusión: Algoritmo de muestreo

- 1) $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2) **para** $t = T, \dots, 1$ **hacer**
 - 3) $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ si $t > 1$ sino $\mathbf{z} = \mathbf{0}$
 - 4) $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5) **fpara**
 - 6) **retorna** $\mathbf{x}_0$
-

6.2.5 Implementación

Los modelos basados en mecanismos de difusión presentados hasta el momento para representar $\epsilon_{\theta}(\mathbf{x}_t, t)$ varían ligeramente dependiendo de la publicación, pero suelen coincidir en la utilización de arquitecturas basadas en U-Net (Ronneberger y otros, 2015) con bloques Wide ResNet (Zagoruyko y Komodakis, 2016) y etapas de autoatención. La representación de t se suele hacer o bien mediante los *embeddings* posicionales típicos de los transformadores (Vaswani y otros, 2017), o bien mediante atributos de Fourier aleatorios (Rahimi y Recht, 2007). No hay ninguna razón que impida que futuras arquitecturas utilicen otras propuestas distintas.

6.2.6 Parametrización de β_t y Σ_θ

Hemos visto que β_t y σ_t^2 controlan la varianza del proceso de difusión directo e inverso, respectivamente. Es habitual utilizar un sistema de distribución lineal para β_t y fijar $\Sigma_\theta = \sigma_t^2 = \beta_t$.

Por ejemplo, en Ho y otros (2020), las varianzas β_t se fijan de modo que sean una secuencia de constantes crecientes desde $\beta_1 = 10^{-4}$ hasta $\beta_T = 0.02$. Son relativamente pequeñas comparadas con el rango normalizado de los valores admisibles para los píxeles, que en dicho trabajo está en el intervalo $[-1,1]$. En dicho trabajo, la varianza del proceso inverso no es un valor paramétrico sino una constante $\Sigma_\theta(\mathbf{x}_t, t) = \sigma_t \mathbf{I}$, donde σ_t es β_t o bien $\tilde{\beta}_t = \frac{1-\alpha_t}{1-\alpha_t} \beta_t$. En dicho artículo las muestras generadas son de alta calidad pero aún no consiguen valores competitivos en lo referente a verosimilitud.

En Nichol y Dhariwal (2021) se proponen varias técnicas para conseguir mejores resultados en lo que a verosimilitud se refiere. Una de dichas mejoras consiste en cambiar la evolución lineal de β_t por una de tipo coseno. El objetivo del cambio no es tanto la función escogida en sí, sino el hecho de conseguir un cambio pequeño en los dos extremos 1 y T y un cambio lineal en los valores intermedios. Para $\Sigma_\theta(\mathbf{x}_t, t)$ proponen interpolar entre los valores β_t y $\tilde{\beta}_t$ mediante la función $\Sigma_\theta(\mathbf{x}_t, t) = \exp(\mathbf{v} \log \beta_t + (1 - \mathbf{v} \log \tilde{\beta}_t))$. Para integrar $\Sigma_\theta(\mathbf{x}_t, t)$ en la función de pérdida, modifican la función original por $L_{\text{hibrida}} = L_{\text{simple}} + \lambda L_{\text{VLB}}$, con $\lambda = 0.001$, utilizando λL_{VLB} para aprender la varianza pero no la media, debido a que comprueban experimentalmente que λL_{VLB} es difícil de optimizar porque tiene gradientes muy variables. Dadas estas circunstancias, para facilitar el proceso de aprendizaje proponen el uso de una media móvil ponderada.

En Kingma y otros (2021) se utiliza un nuevo tipo de parametrización utilizando la razón de señal a ruido (SNR) y se muestra cómo optimizar la reversión minimizando un objetivo variacional.

Mediante la minimización de un objetivo variacional, otros trabajos entrenan σ_t^2 al mismo tiempo que se optimiza el modelo de difusión (Nichol y Dhariwal, 2021) y otros después (Bao y otros, 2022).

6.2.7 Conexión con los VAEs

Los modelos de difusión pueden ser considerados como una forma particular de VAEs jerárquicos. Las diferencias esenciales residen en el hecho que el codificador está prefijado, las dimensiones del espacio latente son las mismas que las de los datos originales, el modelo de eliminación de ruido es el mismo para todos los pasos y que el modelo es entrenado con alguna modificación de los pesos que forman el límite variacional.

6.2.8 Resumen

En este subapartado hemos introducido los modelos probabilísticos de eliminación del ruido por difusión. El modelo se entrena tomando muestras de un proceso de difusión directa a través del entrenamiento de un modelo de reversión capaz de predecir el ruido. Los puntos claves del diseño residen en la arquitectura de red escogida, la ponderación de los objetivos de optimización y la elección de los parámetros de difusión que controlan el proceso (como la distribución del ruido de generación). En Karras y otros (2022) se puede encontrar un compendio de las decisiones de diseño relevantes a la hora de diseñar un modelo de difusión.

6.3 Generación basada en puntuación mediante el uso de ecuaciones diferenciales

6.3.1 Recordatorio de ecuaciones diferenciales

Las ecuaciones diferenciales ordinarias (ODE) tienen la forma siguiente:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

La solución analítica de este tipo de ecuaciones tiene la forma:

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau) d\tau$$

La solución numérica se puede conseguir iterativamente mediante:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t) \Delta t$$

Por otro lado, las ecuaciones diferenciales estocásticas (SDE) toman la forma siguiente:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) + \sigma(\mathbf{x}, t) \omega_t$$

donde a $\mathbf{f}(\mathbf{x}, t)$ se le denomina coeficiente de deriva y a $\sigma(\mathbf{x}, t)$, coeficiente de difusión.

La solución numérica en este caso toma la forma siguiente:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

6.3.2 El proceso de difusión directa como una SDE

Consideremos de nuevo el proceso de difusión directa definido en el subapartado anterior:

$$q(\mathbf{x}_t, \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

Si consideramos pasos infinitesimales podemos expresar a $\mathbf{x}_t$ como:

$$\begin{aligned} \mathbf{x}_t &= \sqrt{1 - \beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(\mathbf{0}, \mathbf{I}) \\ &= \sqrt{1 - \beta(t)\Delta t}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I}) \\ &\text{donde } \beta_t = \beta(t)\Delta t \end{aligned}$$

Aplicando la expansión de Taylor obtenemos:

$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

De la que, llevándola al límite, podemos derivar la ecuación diferencial estocástica (SDE), que describe la difusión en el límite infinitesimal:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)}d\boldsymbol{\omega}_t$$

Atendiendo a la nomenclatura de ecuaciones diferenciales, el término $-\frac{1}{2}\beta(t)\mathbf{x}_t dt$ se correspondería con el término de deriva, y el término $\sqrt{\beta(t)}d\boldsymbol{\omega}_t$ con el término de difusión, que es el que inyecta el ruido.

Una ecuación más general para el SDE utilizada en modelos generativos de difusión sería la siguiente:

$$d\mathbf{x}_t = \mathbf{f}(t)\mathbf{x}_t dt + \mathbf{g}(t)d\boldsymbol{\omega}_t$$

6.3.3 El proceso de difusión inverso mediante SDEs

Visto el proceso directo, ¿cómo se lleva a cabo el proceso inverso? La ecuación para llevar a cabo el proceso inverso generativo de difusión, introducida en Anderson (1982) y propuesta para el proceso de generación de imágenes en Song y otros (2020), es la siguiente:

$$d\mathbf{x}_t = \left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) \right] dt + \sqrt{\beta(t)}d\bar{\omega}_t$$

donde $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ es la función de puntuación, $\sqrt{\beta(t)}d\bar{\omega}_t$ el término de difusión y $-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ representa el término de deriva.

6.3.4 Función de puntuación

Un aspecto importante es definir la forma de determinar la función de puntuación. Una estrategia podría consistir en intentar modelizar dicha función mediante una regresión:

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0,T)} \mathbb{E}_{\mathbf{x}_t \sim q_t(\mathbf{x}_t)} \|s_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)\|_2^2$$

donde t es el paso temporal de difusión, $\mathbf{x}_t$ los datos difundidos, s_{θ} la red neuronal que pretendería modelizar a la función de puntuación y $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ la puntuación marginal de los datos difundidos (Vincent, 2011). El problema es que $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ resulta intratable.

En vez de utilizar el marginal, se puede realizar la difusión de puntos individuales aprovechando que $q_t(\mathbf{x}_t|\mathbf{x}_0)$ es tratable. De esta manera podemos expresar la función objetivo del siguiente modo:

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0,T)} \mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} \mathbb{E}_{\mathbf{x}_t \sim q_t(\mathbf{x}_t)|\mathbf{x}_0} \|s_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|\mathbf{x}_0)\|_2^2$$

Después de aplicar las esperanzas resulta que $s_{\theta}(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$, que es precisamente la función que pretendemos aproximar.

Para la SDE que preserva la varianza sabemos que:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)}d\omega_t$$

$$q_t(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \gamma_t \mathbf{x}_0, \sigma_t^2 \mathbf{I})$$

$$\gamma_t = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$$

$$\sigma_t^2 = 1 - e^{-\int_0^t \beta(s) ds}$$

A modo de detalles de implementación, el muestreo se puede reexpresar del siguiente modo:

$$\begin{aligned} \text{Muestreo:} \quad & \mathbf{x}_t = \gamma_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ \text{Puntuación:} \quad & \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t | \mathbf{x}_0) = \nabla_{\mathbf{x}_t} \frac{(\mathbf{x}_t - \gamma_t \mathbf{x}_0)^2}{2\sigma_t^2} = -\frac{\boldsymbol{\epsilon}}{\sigma_t} \\ \text{Red neuronal modelo:} \quad & s_{\theta}(\mathbf{x}_t, t) := \frac{\boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)}{\sigma_t} \end{aligned}$$

Con lo que la función de optimización se puede expresar del siguiente modo:

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0, T)} \mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \frac{1}{\sigma_t^2} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)\|_2^2$$

Donde la red neuronal alternativa a optimizar para predicción del ruido es $\boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)$.

Una modificación adicional que se puede implementar es introducir una variable $\lambda(t)$ que permita modificar el peso de los distintos constituyentes de la función de pérdida; esto es, de las diferentes partes del proceso de difusión:

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0, T)} \mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \frac{\lambda(t)}{\sigma_t^2} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)\|_2^2$$

Modificando su valor es posible optimizar para la obtención de buenos resultados perceptuales $\lambda(t) = \sigma_t^2$ o bien para la maximización de la verosimilitud $\lambda(t) = \beta(t)$.

Hemos presentado esta posibilidad, pero también es posible usar modelos con parametrizaciones más sofisticadas (Karras y otros, 2022).

Finalmente, en Song y otros (2021) se introducen una serie de modificaciones destinadas a reducir la varianza y mejorar la estabilidad numérica que se traducen en la siguiente propuesta de función de pérdida:

$$\min_{\theta} \mathbb{E}_{t \sim r(t)} \mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \frac{1}{r(t)} \frac{\lambda(t)}{\sigma_t^2} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)\|_2^2$$

donde $r(t)$ es una función de muestreo de la importancia, $r(t) \propto \frac{\lambda(t)}{\sigma_t^2}$. Para detalles concretos de implementación consultad la bibliografía.

6.4 Conclusiones

En este apartado hemos brindado una introducción a los modelos de difusión, los cuales tienen la capacidad de revertir el proceso de difusión directa

y recuperar la imagen original que ha sido degradada. Hemos explorado dos enfoques para modelar esta reversión: los modelos probabilísticos y los basados en ecuaciones diferenciales. No obstante, es importante destacar que el campo de los modelos de difusión es mucho más amplio y complejo que lo que se ha presentado en este apartado, y abarca muchos otros aspectos que no hemos podido cubrir en su totalidad. Este tema es uno de los más relevantes en el ámbito de los modelos generativos, y está en constante evolución y expansión. Para poder profundizar en este tema de manera exhaustiva, sería necesario dedicarle una asignatura completa, y aun así sería difícil abarcar todos los aspectos y avances que se han producido. El objetivo de esta introducción es sentar las bases para que los estudiantes interesados puedan adentrarse en este complejo espacio de conocimiento de manera autónoma y continuar explorando las múltiples posibilidades que ofrece el uso de modelos de difusión en la generación de imágenes.

Resumen

Un modelo generativo es un tipo de modelo de aprendizaje automático que tiene la capacidad de crear nuevos datos a partir de un conjunto de datos de entrenamiento. Estos modelos son ampliamente utilizados en diversas aplicaciones, como la generación de imágenes, la síntesis de voz y la creación de música. A través de la generación de nuevos datos, los modelos generativos pueden ser utilizados para realizar tareas como la simulación, la predicción y la exploración de nuevas posibilidades. En general, los modelos generativos están en constante evolución y expansión, lo que hace que sea un tema de gran interés en el ámbito de la inteligencia artificial y el aprendizaje automático.

En este módulo hemos presentado varios tipos de modelos generativos, incluyendo las redes de generación adversarias, los autocodificadores variacionales y los modelos basados en mecanismos de difusión. Los modelos generativos de texto se tratan en un módulo separado, el de transformadores.

El objetivo de la bibliografía presentada en este módulo es servir como punto de partida para aquellos estudiantes que deseen ampliar sus conocimientos en este campo, aunque no pretende ser exhaustiva. Es importante tener en cuenta que los conocimientos presentados en este módulo están sujetos a una continua investigación y evolución, por lo que es necesaria una constante actualización.

El objetivo ha sido presentar los conocimientos fundamentales que sirven como base para entender los refinamientos introducidos en las publicaciones más recientes.

Glosario

Generative Adversarial Networks (GAN) Término en inglés para referirse a las redes adversarias generativas.

Variational Autoencoders (VAE) Término inglés para referirse a los autocodificadores variacionales.

Bibliografía

- Abdal, R.; Zhu, P.; Mitra, N. J.; Wonka, P.** (2021). «Styleflow: Attribute-conditioned exploration of stylegan-generated images using conditional continuous normalizing flows». *ACM Transactions on Graphics (ToG)*, volumen 40 (n.º 3, págs. 1–21).
- Ackley, D. H.; Hinton, G. E.; Sejnowski, T. J.** (1985). «A learning algorithm for Boltzmann machines». *Cognitive science*, volumen 9 (n.º 1, págs. 147–169).
- Ali, S. M.; Silvey, S. D.** (1966). «A general class of coefficients of divergence of one distribution from another». *Journal of the Royal Statistical Society: Series B (Methodological)*, volumen 28 (n.º 1, págs. 131–142).
- Anderson, B. D.** (1982). «Reverse-time diffusion equation models». *Stochastic Processes and their Applications*, volumen 12 (n.º 3, págs. 313–326).
- Arjovsky, M.; Chintala, S.; Bottou, L.** (2017). «Wasserstein generative adversarial networks». En: «International conference on machine learning» (págs. 214–223). PMLR.
- Bao, F.; Li, C.; Zhu, J.; Zhang, B.** (2022). «Analytic-DPM: an analytic estimate of the optimal reverse variance in diffusion probabilistic models». *arXiv preprint arXiv:2201.06503*.
- Bowman, S. R.; Vilnis, L.; Vinyals, O.; Dai, A. M.; Jozefowicz, R.; Bengio, S.** (2015). «Generating sentences from a continuous space». *arXiv preprint arXiv:1511.06349*.
- Brock, A.; Donahue, J.; Simonyan, K.** (2018). «Large scale GAN training for high fidelity natural image synthesis». *arXiv preprint arXiv:1809.11096*.
- Brock, A.; Lim, T.; Ritchie, J. M.; Weston, N.** (2016). «Neural photo editing with introspective adversarial networks». *arXiv preprint arXiv:1609.07093*.
- Broniatowski, M.** (2014). «Minimum divergence estimators, maximum likelihood and exponential families». *Statistics & Probability Letters*, volumen 93 (págs. 27–33).
- Burda, Y.; Grosse, R.; Salakhutdinov, R.** (2015). «Importance weighted autoencoders». *arXiv preprint arXiv:1509.00519*.
- Burgess, C. P.; Higgins, I.; Pal, A.; Matthey, L.; Watters, N.; Desjardins, G.; Lerchner, A.** (2018). «Understanding disentangling in beta-VAE». *arXiv preprint arXiv:1804.03599*.
- Chen, J.; Sathe, S.; Aggarwal, C.; Turaga, D.** (2017). «Outlier detection with autoencoder ensembles». En: «Proceedings of the 2017 SIAM international conference on data mining» (págs. 90–98). SIAM.
- Chen, P.; Chen, G.; Zhang, S.** (2018a). «Log hyperbolic cosine loss improves variational auto-encoder».
- Chen, R. T.; Li, X.; Grosse, R. B.; Duvenaud, D. K.** (2018b). «Isolating sources of disentanglement in variational autoencoders». *Advances in neural information processing systems*, volumen 31.
- Chen, T.; Zhai, X.; Ritter, M.; Lucic, M.; Houlsby, N.** (2019). «Self-supervised gans via auxiliary rotation loss». En: «Proceedings of the IEEE/CVF conference on computer vision and pattern recognition» (págs. 12.154–12.163).
- Choi, J.; Lee, J.; Shin, C.; Kim, S.; Kim, H.; Yoon, S.** (2022). «Perception Prioritized Training of Diffusion Models». En: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (págs. 11.472–11.481).
- Cohen, G.; Giryes, R.** (2022). «Generative Adversarial Networks». *arXiv preprint arXiv:2203.00667*.
- Crammer, K.; Singer, Y.** (2001). «On the algorithmic implementation of multiclass kernel-based vector machines». *Journal of machine learning research*, volumen 2 (n.º Dec, págs. 265–292).

- Creswell, A.; White, T.; Dumoulin, V.; Arulkumaran, K.; Sengupta, B.; Bharath, A. A.** (2018). «Generative adversarial networks: An overview». *IEEE signal processing magazine*, volumen 35 (n.º 1, págs. 53–65).
- De Vries, H.; Strub, F.; Mary, J.; Larochelle, H.; Pietquin, O.; Courville, A. C.** (2017). «Modulating early visual processing by language». *Advances in Neural Information Processing Systems*, volumen 30.
- Dumoulin, V.; Visin, F.** (2016). «A guide to convolution arithmetic for deep learning». *arXiv preprint arXiv:1603.07285*.
- Dupont, E.** (2018). «Learning disentangled joint continuous and discrete representations». *Advances in Neural Information Processing Systems*, volumen 31.
- Dziugaite, G. K.; Roy, D. M.; Ghahramani, Z.** (2015). «Training generative neural networks via maximum mean discrepancy optimization». *arXiv preprint arXiv:1505.03906*.
- Fedus, W.; Goodfellow, I.; Dai, A. M.** (2018). «Maskgan: better text generation via filling in the _». *arXiv preprint arXiv:1801.07736*.
- Gershman, S.; Goodman, N.** (2014). «Amortized inference in probabilistic reasoning». En: «Proceedings of the annual meeting of the cognitive science society», volumen 36.
- Gneiting, T.; Raftery, A. E.** (2007). «Strictly proper scoring rules, prediction, and estimation». *Journal of the American statistical Association*, volumen 102 (n.º 477, págs. 359–378).
- Gondara, L.** (2016). «Medical image denoising using convolutional denoising autoencoders». En: «2016 IEEE 16th international conference on data mining workshops (ICDMW)» (págs. 241–246). IEEE.
- Goodfellow, I.; Pouget-Abadie, J.; Mirza, M.; Xu, B.; Warde-Farley, D.; Ozair, S.; Courville, A.; Bengio, Y.** (2020). «Generative adversarial networks». *Communications of the ACM*, volumen 63 (n.º 11, págs. 139–144).
- Guo, J.; Lu, S.; Cai, H.; Zhang, W.; Yu, Y.; Wang, J.** (2018). «Long text generation via adversarial training with leaked information». En: «Proceedings of the AAAI conference on artificial intelligence», volumen 32.
- Hartmann, K. G.; Schirrmeister, R. T.; Ball, T.** (2018). «EEG-GAN: Generative adversarial networks for electroencephalographic (EEG) brain signals». *arXiv preprint arXiv:1806.01875*.
- Heusel, M.; Ramsauer, H.; Unterthiner, T.; Nessler, B.; Hochreiter, S.** (2017). «Gans trained by a two time-scale update rule converge to a local nash equilibrium». *Advances in neural information processing systems*, volumen 30.
- Higgins, I.; Matthey, L.; Pal, A.; Burgess, C.; Glorot, X.; Botvinick, M.; Mohamed, S.; Lerchner, A.** (2016). «beta-VAE: Learning basic visual concepts with a constrained variational framework».
- Hinton, G. E.; Zemel, R.** (1993). «Autoencoders, minimum description length and Helmholtz free energy». *Advances in neural information processing systems*, volumen 6.
- Hitchcock, F. L.** (1941). «The distribution of a product from several sources to numerous localities». *Journal of mathematics and physics*, volumen 20 (n.º 1-4, págs. 224–230).
- Ho, J.; Jain, A.; Abbeel, P.** (2020). «Denoising diffusion probabilistic models». *Advances in Neural Information Processing Systems*, volumen 33 (págs. 6840–6851).
- Hoffman, J.; Tzeng, E.; Park, T.; Zhu, J.-Y.; Isola, P.; Saenko, K.; Efros, A.; Darrell, T.** (2018). «Cycada: Cycle-consistent adversarial domain adaptation». En: «International conference on machine learning» (págs. 1989–1998). PMLR.
- Hou, X.; Shen, L.; Sun, K.; Qiu, G.** (2017). «Deep feature consistent variational auto-encoder». En: «2017 IEEE winter conference on applications of computer vision (WACV)» (págs. 1133–1141). IEEE.
- Huang, X.; Belongie, S.** (2017). «Arbitrary style transfer in real-time with adaptive instance normalization». En: «Proceedings of the IEEE international conference on computer vision» (págs. 1501–1510).
- Ioffe, S.; Szegedy, C.** (2015). «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: «International conference on machine learning» (págs. 448–456). PMLR.

- Isola, P.; Zhu, J.-Y.; Zhou, T.; Efros, A. A.** (2017). «Image-to-image translation with conditional adversarial networks». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 1125–1134).
- Jang, E.; Gu, S.; Poole, B.** (2016). «Categorical reparameterization with gumbel-softmax». *arXiv preprint arXiv:1611.01144*.
- Jetchev, N.; Bergmann, U.; Vollgraf, R.** (2016). «Texture synthesis with spatial generative adversarial networks». *arXiv preprint arXiv:1611.08207*.
- Karras, T.; Aila, T.; Laine, S.; Lehtinen, J.** (2017). «Progressive growing of GANs for improved quality, stability, and variation». *arXiv preprint arXiv:1710.10196*.
- Karras, T.; Aittala, M.; Aila, T.; Laine, S.** (2022). «Elucidating the Design Space of Diffusion-Based Generative Models». *arXiv preprint arXiv:2206.00364*.
- Karras, T.; Laine, S.; Aila, T.** (2019). «A style-based generator architecture for generative adversarial networks». En: «Proceedings of the IEEE/CVF conference on computer vision and pattern recognition» (págs. 4401–4410).
- Kingma, D.; Salimans, T.; Poole, B.; Ho, J.** (2021). «Variational diffusion models». *Advances in neural information processing systems*, volumen 34 (págs. 21.696–21.707).
- Kingma, D. P.; Salimans, T.; Jozefowicz, R.; Chen, X.; Sutskever, I.; Welling, M.** (2016). «Improved variational inference with inverse autoregressive flow». *Advances in neural information processing systems*, volumen 29.
- Kingma, D. P.; Welling, M.** (2013). «Auto-encoding variational bayes». *arXiv preprint arXiv:1312.6114*.
- Kobyzev, I.; Prince, S. J.; Brubaker, M. A.** (2020). «Normalizing flows: An introduction and review of current methods». *IEEE transactions on pattern analysis and machine intelligence*, volumen 43 (n.º 11, págs. 3964–3979).
- Koller, D.; Friedman, N.** (2009). *Probabilistic Graphical Models: Principles and Techniques*. Adaptive computation and machine learning. MIT Press. ISBN 9780262013192. <https://books.google.co.in/books?id=7dzpHCHzNQ4C>.
- Kolouri, S.; Pope, P. E.; Martin, C. E.; Rohde, G. K.** (2018). «Sliced-wasserstein auto-encoder: An embarrassingly simple generative model». *arXiv preprint arXiv:1804.01947*.
- Kullback, S.; Leibler, R. A.** (1951). «On information and sufficiency». *The annals of mathematical statistics*, volumen 22 (n.º 1, págs. 79–86).
- Kumar, A.; Sattigeri, P.; Balakrishnan, A.** (2017). «Variational inference of disentangled latent concepts from unlabeled observations». *arXiv preprint arXiv:1711.00848*.
- Lample, G.; Zeghidour, N.; Usunier, N.; Bordes, A.; Denoyer, L.; Ranzato, M.** (2017). «Fader networks: Manipulating images by sliding attributes». *Advances in neural information processing systems*, volumen 30.
- Larsen, A. B. L.; Sønderby, S. K.; Larochelle, H.; Winther, O.** (2016). «Autoencoding beyond pixels using a learned similarity metric». En: «International conference on machine learning» (págs. 1558–1566). PMLR.
- Ledig, C.; Theis, L.; Huszár, F.; Caballero, J.; Cunningham, A.; Acosta, A.; Aitken, A.; Tejani, A.; Totz, J.; Wang, Z. y otros** (2017). «Photo-realistic single image super-resolution using a generative adversarial network». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 4681–4690).
- Lei Ba, J.; Kiros, J. R.; Hinton, G. E.** (2016). «Layer normalization». *ArXiv e-prints*, (págs. arXiv-1607).
- Luc, P.; Couprie, C.; Chintala, S.; Verbeek, J.** (2016). «Semantic segmentation using adversarial networks». *arXiv preprint arXiv:1611.08408*.
- Maaløe, L.; Sønderby, C. K.; Sønderby, S. K.; Winther, O.** (2016). «Auxiliary deep generative models». En: «International conference on machine learning» (págs. 1445–1453). PMLR.
- Maas, A. L.; Hannun, A. Y.; Ng, A. Y. y otros** (2013). «Rectifier nonlinearities improve neural network acoustic models». En: «Proc. icml», volumen 30 (pág. 3). Citeseer.
- Makhzani, A.; Frey, B.** (2013). «K-sparse autoencoders». *arXiv preprint arXiv:1312.5663*.
- Maleki, A.; Do, T.** (2000). «Review of probability theory». *CS*, volumen 229 (n.º 2, pág. 1).

- Mescheder, L.; Geiger, A.; Nowozin, S.** (2018). «Which training methods for GANs do actually converge?». En: «International conference on machine learning» (págs. 3481–3490). PMLR.
- Metropolis, N.; Ulam, S.** (1949). «The Monte Carlo method». *Journal of the American statistical association*, volumen 44 (n.º 247, págs. 335–341).
- Michelucci, U.** (2022). «An Introduction to Autoencoders». *arXiv preprint arXiv:2201.03898*.
- Mirza, M.; Osindero, S.** (2014). «Conditional generative adversarial nets». *arXiv preprint arXiv:1411.1784*.
- Miyato, T.; Kataoka, T.; Koyama, M.; Yoshida, Y.** (2018). «Spectral normalization for generative adversarial networks». *arXiv preprint arXiv:1802.05957*.
- Miyato, T.; Koyama, M.** (2018). «cGANs with projection discriminator». *arXiv preprint arXiv:1802.05637*.
- Moghadam, M. M.; Boroomand, B.; Jalali, M.; Zareian, A.; DaeiJavad, A.; Manshaei, M. H.** (2021). «Game of GANs: Game Theoretical Models for Generative Adversarial Networks». *arXiv preprint arXiv:2106.06976*.
- Ng, A. y otros** (2011). «Sparse autoencoder». *CS294A Lecture notes*, volumen 72 (n.º 2011, págs. 1–19).
- Nguyen, X.; Wainwright, M. J.; Jordan, M. I.** (2010). «Estimating divergence functionals and the likelihood ratio by convex risk minimization». *IEEE Transactions on Information Theory*, volumen 56 (n.º 11, págs. 5847–5861).
- Nichol, A. Q.; Dhariwal, P.** (2021). «Improved denoising diffusion probabilistic models». En: «International Conference on Machine Learning» (págs. 8162–8171). PMLR.
- Nowozin, S.; Cseke, B.; Tomioka, R.** (2016). «f-gan: Training generative neural samplers using variational divergence minimization». *Advances in neural information processing systems*, volumen 29.
- Odena, A.** (2016). «Semi-supervised learning with generative adversarial networks». *arXiv preprint arXiv:1606.01583*.
- Pang, G.; Shen, C.; Cao, L.; Hengel, A. V. D.** (2021). «Deep learning for anomaly detection: A review». *ACM Computing Surveys (CSUR)*, volumen 54 (n.º 2, págs. 1–38).
- Park, S. W.; Kwon, J.** (2020). «SphereGAN: Sphere generative adversarial network based on geometric moment matching and its applications». *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Patashnik, O.; Wu, Z.; Shechtman, E.; Cohen-Or, D.; Lischinski, D.** (2021). «Styleclip: Text-driven manipulation of stylegan imagery». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 2085–2094).
- Plaut, E.** (2018). «From principal subspaces to principal components with linear autoencoders». *arXiv preprint arXiv:1804.10253*.
- Pulgar, F. J.; Charle, F.; Rivera, A. J.; Del Jesus, M. J.** (2018). «AEkNN: An AutoEncoder kNN-based classifier with built-in dimensionality reduction». *arXiv preprint arXiv:1802.08465*.
- Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J. y otros** (2021). «Learning transferable visual models from natural language supervision». En: «International Conference on Machine Learning» (págs. 8748–8763). PMLR.
- Radford, A.; Metz, L.; Chintala, S.** (2015). «Unsupervised representation learning with deep convolutional generative adversarial networks». *arXiv preprint arXiv:1511.06434*.
- Rahimi, A.; Recht, B.** (2007). «Random features for large-scale kernel machines». *Advances in neural information processing systems*, volumen 20.
- Rainforth, T.; Kosiorek, A.; Le, T. A.; Maddison, C.; Igl, M.; Wood, F.; Teh, Y. W.** (2018). «Tighter variational bounds are not necessarily better». En: «International Conference on Machine Learning» (págs. 4277–4285). PMLR.
- Ramesh, A.; Pavlov, M.; Goh, G.; Gray, S.; Voss, C.; Radford, A.; Chen, M.; Sutskever, I.** (2021). «Zero-shot text-to-image generation». En: «International Conference on Machine Learning» (págs. 8821–8831). PMLR.

Reid, M.; Williamson, R. y otros (2011). «Information, divergence and risk for binary experiments».

Rezende, D.; Mohamed, S. (2015). «Variational inference with normalizing flows». En: «International conference on machine learning» (págs. 1530–1538). PMLR.

Rifai, S.; Mesnil, G.; Vincent, P.; Muller, X.; Bengio, Y.; Dauphin, Y.; Glorot, X. (2011). «Higher order contractive auto-encoder». En: «Joint European conference on machine learning and knowledge discovery in databases» (págs. 645–660). Springer.

Ronneberger, O.; Fischer, P.; Brox, T. (2015). «U-net: Convolutional networks for biomedical image segmentation». En: «International Conference on Medical image computing and computer-assisted intervention» (págs. 234–241). Springer.

Salimans, T. (2016). «A structured variational auto-encoder for learning deep hierarchies of sparse features». *arXiv preprint arXiv:1602.08734*.

Salimans, T.; Kingma, D. P. (2016). «Weight normalization: A simple reparameterization to accelerate training of deep neural networks». *Advances in neural information processing systems*, volumen 29.

Shaham, T. R.; Dekel, T.; Michaeli, T. (2019). «Singan: Learning a generative model from a single natural image». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 4570–4580).

Snell, J.; Ridgeway, K.; Liao, R.; Roads, B. D.; Mozer, M. C.; Zemel, R. S. (2017). «Learning to generate images with perceptual similarity metrics». En: «2017 IEEE International Conference on Image Processing (ICIP)» (págs. 4277–4281). IEEE.

Soch, J.; Faulkenberry, T. J.; Petrykowski, K.; Allefeld, C. (2020). *The Book of Statistical Proofs*. Zenodo. doi: 10.5281/zenodo.4305950. <https://doi.org/10.5281/zenodo.4305950>.

Sohl-Dickstein, J.; Weiss, E.; Maheswaranathan, N.; Ganguli, S. (2015). «Deep unsupervised learning using nonequilibrium thermodynamics». En: «International Conference on Machine Learning» (págs. 2256–2265). PMLR.

Sohn, K.; Lee, H.; Yan, X. (2015). «Learning structured output representation using deep conditional generative models». *Advances in neural information processing systems*, volumen 28.

Song, Y.; Durkan, C.; Murray, I.; Ermon, S. (2021). «Maximum likelihood training of score-based diffusion models». *Advances in Neural Information Processing Systems*, volumen 34 (págs. 1415–1428).

Song, Y.; Sohl-Dickstein, J.; Kingma, D. P.; Kumar, A.; Ermon, S.; Poole, B. (2020). «Score-based generative modeling through stochastic differential equations». *arXiv preprint arXiv:2011.13456*.

Sriperumbudur, B. K.; Gretton, A.; Fukumizu, K.; Schölkopf, B.; Lanckriet, G. R. (2010). «Hilbert space embeddings and metrics on probability measures». *The Journal of Machine Learning Research*, volumen 11 (págs. 1517–1561).

Subramanian, A. (2020). «PyTorch-VAE». <https://github.com/AntixK/PyTorch-VAE>.

Tolstikhin, I.; Bousquet, O.; Gelly, S.; Schoelkopf, B. (2017). «Wasserstein auto-encoders». *arXiv preprint arXiv:1711.01558*.

Van Den Oord, A.; Vinyals, O. y otros (2017). «Neural discrete representation learning». *Advances in neural information processing systems*, volumen 30.

Van Loan, C. F.; Golub, G. (1996). «Matrix computations (Johns Hopkins studies in mathematical sciences)». *Matrix Computations*.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I. (2017). «Attention is all you need». *Advances in neural information processing systems*, volumen 30.

Vincent, P. (2011). «A connection between score matching and denoising autoencoders». *Neural computation*, volumen 23 (n.º 7, págs. 1661–1674).

Vincent, P.; Larochelle, H.; Bengio, Y.; Manzagol, P.-A. (2008). «Extracting and composing robust features with denoising autoencoders». En: «Proceedings of the 25th international conference on Machine learning» (págs. 1096–1103).

- Vincent, P.; Larochelle, H.; Lajoie, I.; Bengio, Y.; Manzagol, P.-A.; Bottou, L.** (2010). «Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion». *Journal of machine learning research*, volumen 11 (n.º 12).
- Wang, T.-C.; Liu, M.-Y.; Zhu, J.-Y.; Tao, A.; Kautz, J.; Catanzaro, B.** (2018). «High-resolution image synthesis and semantic manipulation with conditional GANs». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 8798–8807).
- Wu, H.; Zheng, S.; Zhang, J.; Huang, K.** (2019). «GP-GAN: Towards realistic high-resolution image blending». En: «Proceedings of the 27th ACM international conference on multimedia» (págs. 2487–2495).
- Xia, W.; Zhang, Y.; Yang, Y.; Xue, J.-H.; Zhou, B.; Yang, M.-H.** (2022). «Gan inversion: A survey». *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Zagoruyko, S.; Komodakis, N.** (2016). «Wide residual networks». *arXiv preprint arXiv:1605.07146*.
- Zhang, H.; Goodfellow, I.; Metaxas, D.; Odena, A.** (2019). «Self-attention generative adversarial networks». En: «International conference on machine learning» (págs. 7354–7363). PMLR.
- Zhao, S.; Song, J.; Ermon, S.** (2017). «Infovae: Information maximizing variational autoencoders». *arXiv preprint arXiv:1706.02262*.
- Zhou, C.; Paffenroth, R. C.** (2017). «Anomaly detection with robust deep autoencoders». En: «Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining» (págs. 665–674).

